

Общая информация

Длительность	3 недели
Семестр	2 семестр
Аттестация	зачет

Авторы и преподаватели

Дмитрий Коробченко

- Nvidia, Deep Learning R&D-инженер.
- Исследователь и разработчик в областях машинного обучения, нейронных сетей, компьютерного зрения и обработки сигналов.
- Занимался задачами, связанными с медицинскими изображениями, распознаванием, обработкой изображений, 3D-анимацией.
- Читает вводные научно-популярные лекции по Deep Learning.
- Преподаватель Skillfactory по нейронным сетям, AI-разработке, ML и DL.

Константин Некрасов

- АО «Газпромбанк», директор по разработке ML-моделей.
- Ведёт проекты в области машинного обучения для решения прикладных задач финтеха.
- Эксперт в глубоком обучении, в частности в направлениях обработки естественного языка (NLP).
- Разрабатывает и внедряет ML-модели в продуктовую среду.
- Преподаёт дисциплины по Data Science, Data Engineering, Machine Learning, DevOps и Web Development в ведущих вузах страны: МИФИ, МФТИ, ВШЭ.

О дисциплине

Дисциплина «Продвинутые методы глубокого обучения» направлена на углублённое изучение современных архитектур нейронных сетей и методов их обучения. В курсе рассматриваются свёрточные нейронные сети (CNN), методы оптимизации, регуляризации и ускорения обучения, а также подходы transfer learning и fine-tuning.

Особое внимание уделяется практическому применению моделей в задачах компьютерного зрения: от построения базовых CNN до использования предобученных архитектур. В рамках дисциплины студенты последовательно проходят путь от понимания свёрток и архитектур до работы с пространствами признаков и поиском похожих объектов.

Курс завершается практическим проектом, в котором студенты сравнивают различные подходы к классификации данных (MLP, CNN и transfer learning), что позволяет закрепить как теоретические знания, так и навыки построения моделей.

Цель дисциплины

- Сформировать у студентов системное понимание продвинутых методов глубокого обучения.
- Развить практические навыки построения, обучения и анализа нейронных сетей для задач классификации.

В рамках дисциплины вы:

- изучите принципы работы свёрточных нейронных сетей и их архитектурные компоненты (Conv, Pooling, Flatten, Dense);
- разберётесь в методах оптимизации нейросетей, включая SGD, Adam, batch-нормализацию и регуляризацию;
- поймёте влияние гиперпараметров (learning rate, инициализация весов и так далее) на обучение моделей;
- освоите подходы transfer learning и fine-tuning с использованием предобученных моделей;
- научитесь работать с представлениями данных и решать задачи поиска похожих объектов;
- реализуете и обучите собственные модели (CNN и EfficientNet) для задачи классификации;
- проведёте сравнительный анализ моделей и научитесь интерпретировать полученные результаты;
- получите опыт оформления результатов в виде воспроизводимого эксперимента (ноутбук с кодом и выводами).

Содержание дисциплины

Неделя	Название модуля	Содержание
1	Модуль 1. Свёрточные нейронные сети	► Введение в свёртки ► Как применять нейросеть ► Операция свёртки ► Простой свёрточный слой ► Практика. Параметры Conv2D слоя ► Усложняем свёрточный слой ► Практика. Реализация простой свёрточной сети ► Пулинг-слой ► Практика. Добавление Pooling-слоя ► Первая свёрточная сеть ► Практика. Добавление Flatten- и Dense-слоёв. Обучение модели ► Inception V3
2	Модуль 2. Задача оптимизации	► Функции активации ► Инициализация весов ► Влияние learning rate и масштаба признаков на сходимость ► Batch-нормализация ► Dropout-регуляризация ► Производная функции ► Стохастический градиентный спуск (SGD) ► Adam — Adaptive Moment Estimation ► Матричные операции
3	Модуль 3. Finetuning & Transfer learning	► Как получить такие картинки ► Перенос обучения ► Fine-tuning ► Автокодировщики ► Un-pooling ► Поиск похожих изображений ► Быстрый KNN ► Пространство представлений ► Практика

Практика

Практическая подготовка по дисциплине осуществляется в формате синхронных онлайн-занятий и асинхронной самостоятельной работы на образовательной платформе.

Асинхронная практика включает интерактивные задания: тесты (выбор одного или нескольких ответов), задания с вводом ответа.

Синхронные занятия проводятся в формате практико-ориентированных онлайн-семинаров с разбором прикладных кейсов.

- По завершении дисциплины выполняется семестровое задание — итоговое тестирование. Срок выполнения — до трёх недель после окончания дисциплины. Во время зачётной недели задание сдать уже нельзя.
- Итогом дисциплины является проект (сессионное задание) — кейс «Классификация музыкальных жанров: сравнение MLP, CNN и EfficientNet». Проект сдаётся в течение двух недель после завершения дисциплины либо в период зачётной недели.

Оценивание

Оценка за дисциплину суммируется из оценок за выполнение итогового тестирования (максимальное количество баллов — 50) и сессионного задания, за которое вы также сможете получить максимально 50 баллов.

Для получения зачёта вам необходимо набрать 60 баллов и выше.

Зачет	Оценка по пятибалльной шкале	Сумма баллов по дисциплине	Оценка (ECTS)
Зачтено	5 (отлично)	100–90	A
		89–85	B
	4 (хорошо)	84–75	C
		74–70	D
	3 (удовлетворительно)	69–65	E
		64–60	
Не зачтено		59–0	F

1/13

Введение в свёртки

текущий урок



В этом модуле мы поговорим про свёрточные сети.

Задачи, которые мы научимся решать:

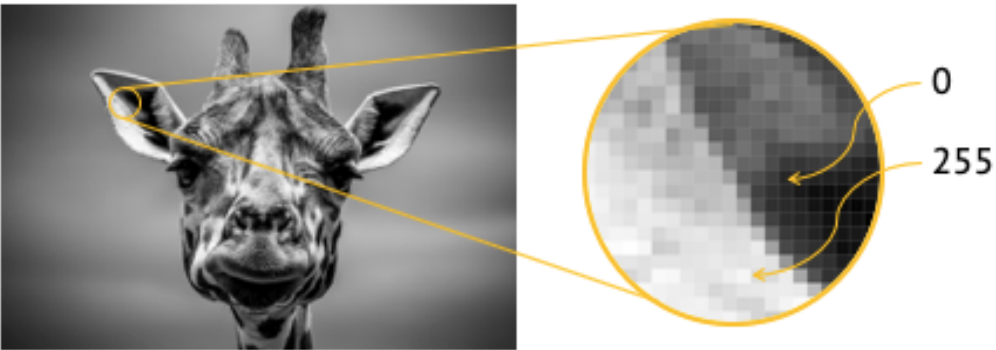
- ▶ Задачи классификации.
Например, понять какой объект изображен на картинке.
- ▶ Задачи регрессии.
Например, определить возраст человека на картинке.

Итак, приступим.

Как компьютер воспринимает картинку?

Любая картинка состоит из пикселей, а в каждом пикселе закодирована яркость — это число от 0 до 255. Яркость 0 означает черный цвет, а значение 255 — белый.

Для примера возьмем картинку и очень сильно её увеличим. После этого станут отчетливо видны пиксели, которые имеют различные яркости.



В данном случае у нас чёрно-белая картинка и пиксели выглядят черными, белыми и серыми. При работе с цветными изображениями всё становится несколько сложнее, потому что яркость будет определена для каждого из 3 цветовых каналов RGB (красного, синего, зеленого).

<

Предыдущий урок

Следующий урок

>

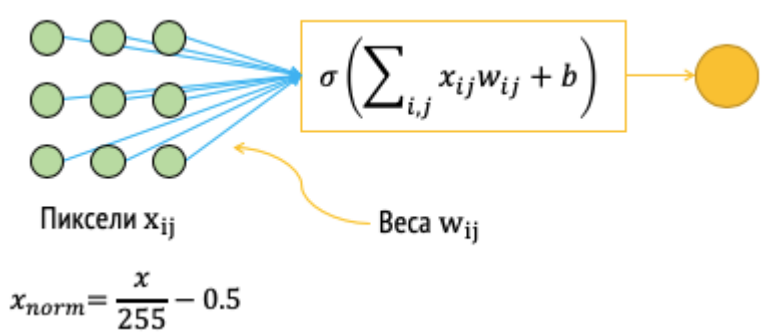
2/13 Как применять нейросеть

текущий урок



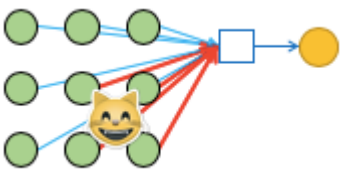
Как применять нейросеть к картинке?

Давайте применим уже знакомый нам многослойный перцептрон. Пусть на вход у нас поступают пиксели картинки, которые образуют собой двумерную матрицу. Сопоставим каждой ячейке нашей пиксельной матрицы определенный вес, зависящий от яркости, а затем сложим все наши элементы матрицы с этими весами и проведем через сигмоиду, как мы это обычно делаем в простом нейроне.

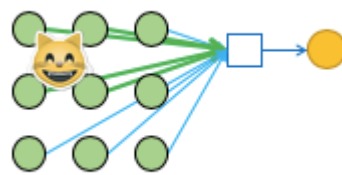


При таком подходе могут возникнуть проблемы:

- 1. Картинка может быть очень большого размера. Допустим, если картинка имеет размеры 300 x 300 то нам нужно 90 000 весов (а это очень много).
- 2. Допустим, мы хотим определять, изображен ли на картинке котик. Возьмем картинку, на которой котик изображен в правом нижнем углу. Во время градиентного спуска немного изменятся веса, которые смотрят непосредственно на этого котика (на этой картинке — красные веса).



Но если мы возьмем другую картинку, на которой котик будет находиться в левом верхнем углу, то у нас при градиентном спуске, немного изменятся зеленые веса, потому что котик будет находиться в другой части картинки.



Таким образом, котик в разных частях картинки приводит к обновлению различных весов, хотя эти веса должны научиться находить на картинке одно и то же животное. Т.е. если бы у нас была выборка картинок только с котиками в правом нижнем углу, мы бы никогда не научились находить котиков на картинках, на которых они расположены в левом верхнем углу.

← Предыдущий урок

Следующий урок →



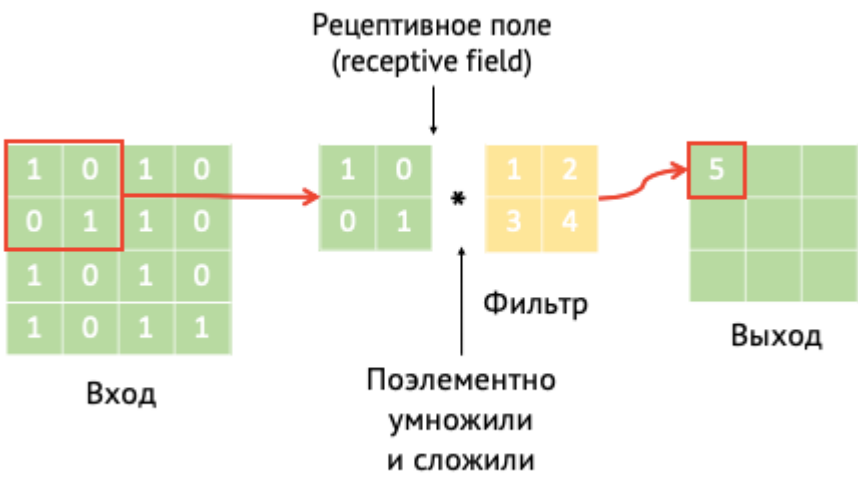
Получается, что рассмотренный способ нам не подходит. Как же быть?

К счастью, для такого рода анализа изображений существует операция свёртки.

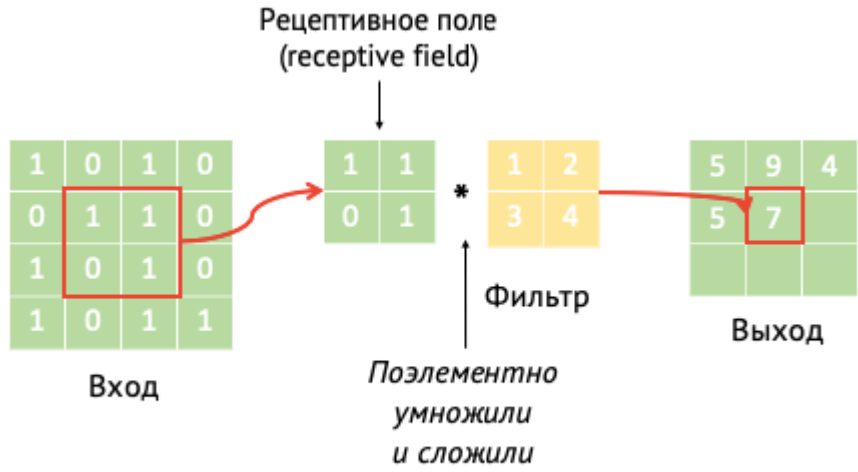
Как работает операция свёртки?

Пусть на вход подается изображение, которое состоит из 0 и 1 (так нам будет удобнее устно считать и понимать, как именно работает свёртка) и пусть имеется скользящее окно фиксированного размера (в нашем случае 2×2). Нашим окошком 2×2 мы пробегаем по изображению и каждый кусочек, по которому пробегает окошко, поэлементно умножаем на некоторые веса и складываем. То есть получаем скалярное произведение нашего кусочка изображения и фильтра, состоящего из весов, которые мы будем обучать.

На картинке ниже представлен первый шаг, результат выполнения скалярного произведения кусочка изображения на фильтр — число 5.



Будем постепенно передвигать окошко и получать некие выходные данные. На картинке ниже показан один из шагов вычислений:



На самом деле, про свёртки известно очень давно. В любом редакторе, например в Photoshop, есть реализации свёрток.

Пример № 1

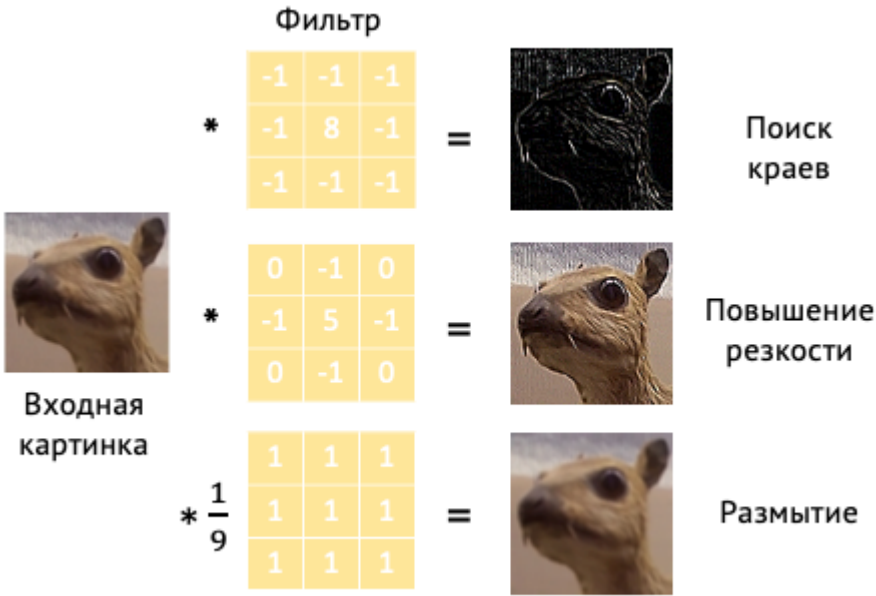
Пусть на вход подается картинка. Она пропускается через фильтр 3×3 , у которого по центру стоит число 8, а во всех остальных ячейках -1 (сумма элементов равна 0).

Если этому фильтру подать на вход изображение, которое является однотонной заливкой, то поэлементно умножив все яркости на коэффициенты и сложив, мы получим 0, который соответствует чёрному цвету. Те места, где заливка не однотонная (есть перепад яркости), будут восприниматься как граница и значение в этих ячейках будет отлично от 0.

В результате свертки получаем картинку с подсвеченными краями.

Пример № 2

Рассмотрим другой фильтр, сумма весов которого равна 1. При однотонной заливке этот фильтр не будет менять цвет. Если заливка не будет однотонной, то на границе объекта на изображении у нас будет повышаться яркость. Визуально это будет восприниматься как увеличение резкости. Существует фильтр и для обратной операции — размытия. Такой фильтр состоит из одинаковых элементов и усредняет все цвета, которые видит.

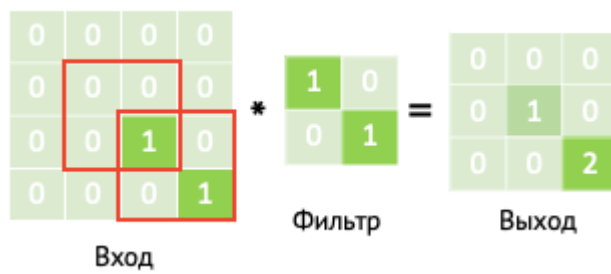




Как реализовать простой свёрточный слой в нейросети?

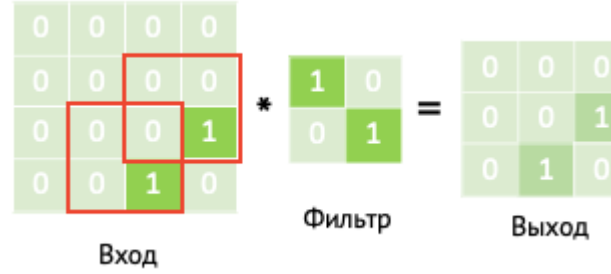
Для начала нужно разобраться, как это поможет нам искать на картинках определённые объекты (кошек, собак, уток).

Рассмотрим пример. Пусть на картинке у нас расположена чёрточка под углом, и нам нужно научиться находить её на картинке. Тогда возьмём в качестве фильтра эту же чёрточку.



Заметим, что после применения операции ненулевые значения на выходе будут только для двух положений скользящего окна (положения обозначены красной рамкой).

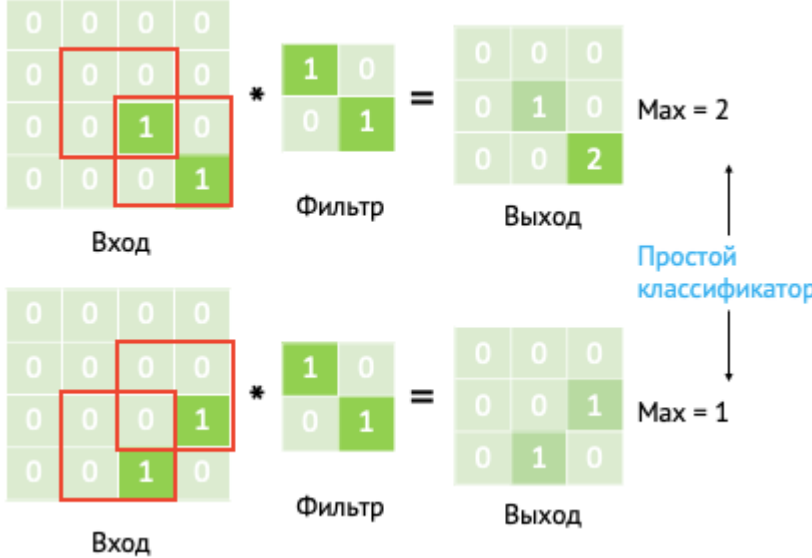
Посмотрим, как будет работать свёртка с картинкой, на которой чёрточка повернута в другую сторону. Применив операцию свёртки, получим немного другие значения: здесь тоже только два ненулевых значения на выходе, но они меньше, чем в прошлый раз.



Это говорит нам о том, что на самом деле мы научились определять, что чёрточка на картинке повернута.

Как же теперь нам понять, куда именно повернута чёрточка?

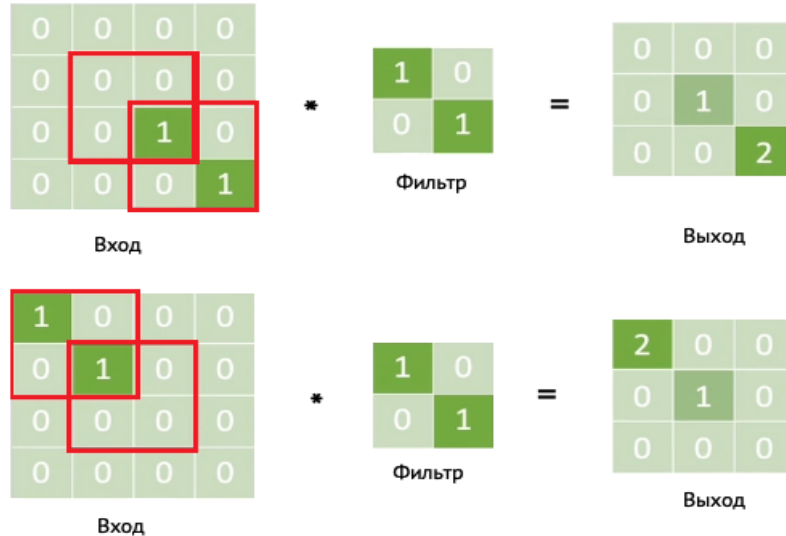
Нужно просто взять максимум функции — именно он и будет определять поворот нашей чёрточки.



Максимум функции — простой классификатор для изображений.

Данный классификатор обладает некоторыми полезными свойствами. Например, свёртка и сдвиг **коммутативны** (если эти операции поменять местами, то результат не изменится).

Посмотрим на примере, что произойдет на выходе, если чёрточка на изображении переместится в другой угол.



Снова получим для обоих случаев лишь два ненулевых положения окна. Причём для обоих случаев результат одинаков (только сдвинут). Это означает, что неважно, что делать вначале, сдвиг или свёртку – результат всё равно будет одинаков.

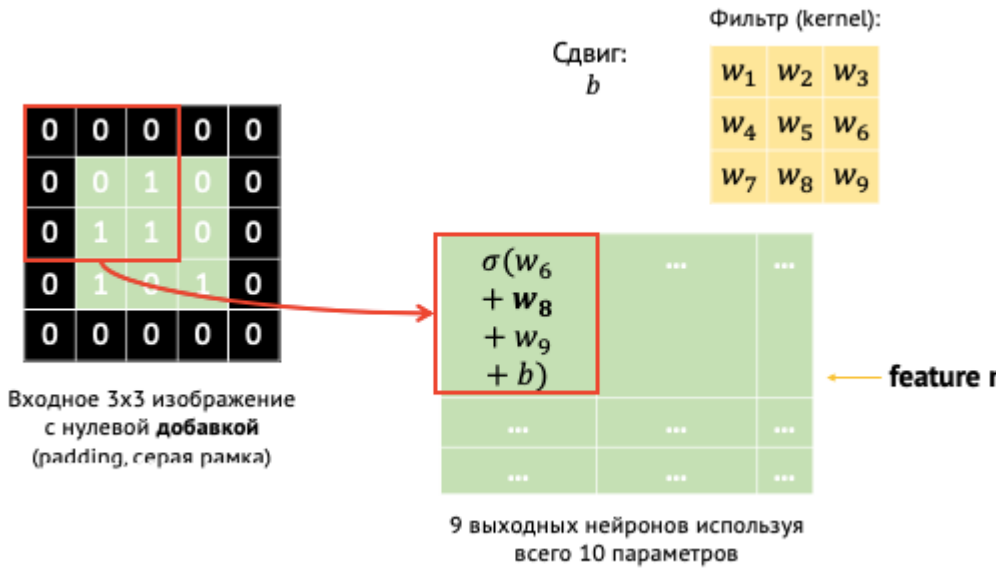
Это важный и полезный результат, так как наш классификатор — максимум одинаков. То есть неважно, где будет расположен котик на картинке (в левом верхнем или в правом нижнем углу), мы всё равно будем знать, что это котик.

Теперь мы можем собрать свёрточный слой в нейросети.

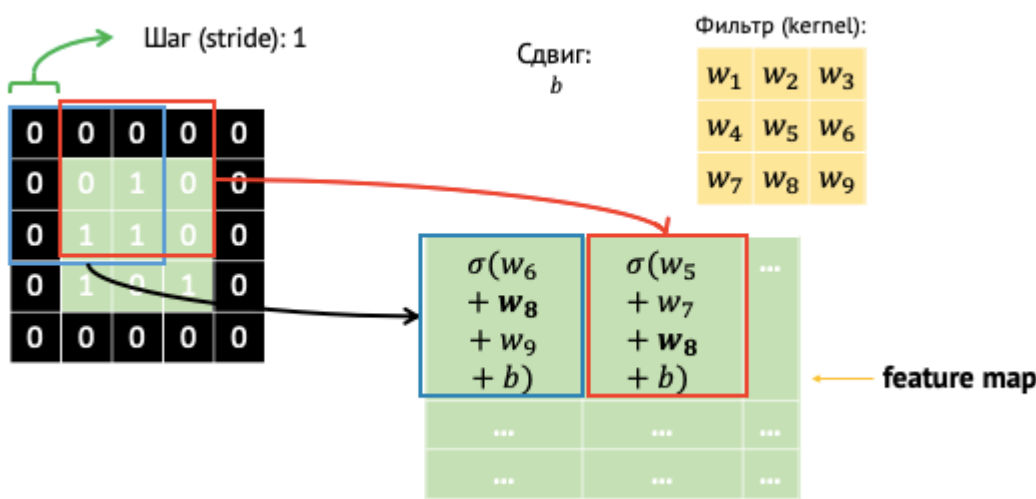
Как это работает?

На вход подаётся зелёная картинка 3 × 3 с некой рамкой по бокам (обычно она заполнена нулями), называемой *padding*, или добавкой, необходимой для того, чтобы мы смогли поместить столько окошек, какого размера картинка, чтобы картинка осталась такого же размера за счёт этой добавки.

У нас есть веса, которые нейросеть будет обучать с помощью градиентного спуска. На картинке ниже показано, как будет посчитан первый нейрон.



Кроме того, у нас есть ещё один параметр — шаг (сдвиг, *stride*), с которым двигается наше скользящее окно. Здесь шаг — 1 пиксель.



Продолжаем так дальше и на выходе получаем также картинку 3 × 3 — результат свёртки, называемый *feature map* (карта фичей).

Заметим, что для этого преобразования мы использовали всего 10 параметров (9 весов и 1 шаг).

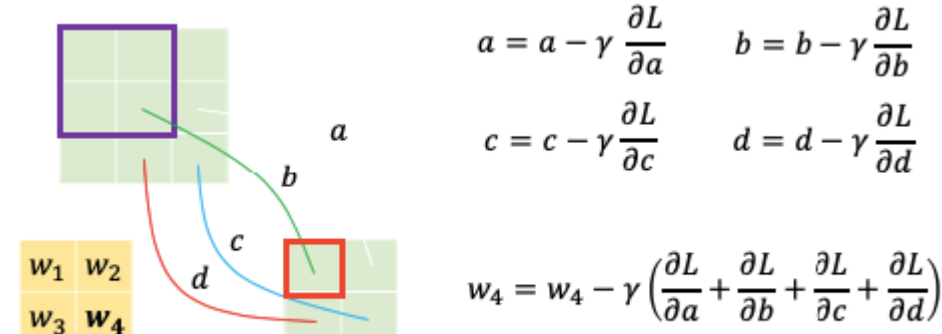
Как работает градиентный спуск для операции свёртки?

На самом деле свёрточный слой — частный случай полносвязного. Веса, находящиеся вне нашего поля обзора, просто занулены. И тогда имеем полносвязный слой, а для полносвязного слоя мы уже умеем считать производную.

Как считать производную для свёрточного слоя?

Возьмём картинку 3 × 3 и фильтр 2 × 2. На этой картинке есть всего четыре возможных положения фильтра. Последим за одним параметром w_4 . Нам интересно взять производную нашей функции потерь именно по этому параметру. Все использования нашего параметра в сети назовём различными буквами (a , b , c , d) и будем считать по ним производную.

Чтобы сделать шаг по градиенту, необходимо будет посчитать производную потери по каждой из этих букв. Сдвинем каждый параметр по направлению антиградиента, но если вспомнить что a , b , c , d — это не разные параметры, а один и тот же, то становится понятно, как на самом деле работает градиентный спуск для свёртки: мы четыре раза обновили один и тот же параметр и обновили его на сумму градиентов по всем использованиям на нашей картинке.



Ещё одним плюсом свёрточного слоя является то, что для его реализации нужно очень мало параметров. Пусть у нас имеется картинка 300 × 300, и на выходе мы хотим получить картинку такого же размера. В случае с полносвязным слоем нам необходимо 8.1 * 10⁹ параметров, в то время как в случае со свёрточным слоем и окном 5 × 5 — всего 25 параметров.

Практика. Параметры Conv2D слоя

В данном видео мы рассмотрим построение сверточной нейронной сети с использованием Keras, начнем с добавлении сверточного слоя в модель. Применим полученные знания к задаче классификации FashionMNIST.



Ключевые тезисы:

- ▶ Перед началом работы нужно войти в свой Google Account и [загрузить ноутбук с платформы Skillfactory](#).
- ▶ В Keras мы можем построить сверточную сеть, добавив сверточный слой. Для примера рассмотрим задачу FashionMNIST.
- ▶ Перед началом работы необходимо проверить наличие и правильную версию TensorFlow, импортировать необходимые зависимости и проверить работу GPU.
- ▶ Набор данных FashionMNIST содержит изображения размером 28x28, разделенные на части Train (60 000 изображений) и Validation (10 000 изображений) с 10 классами.
- ▶ Каждый пиксель представляет оттенок серого, а классы представлены в категориальном виде.
- ▶ Для построения модели начнем с добавления первого сверточного слоя Conv2D с параметрами: количество фильтров, размер ядра, `stride` и `padding`.
- ▶ Необходимо задать размер входа для расчета следующих размерностей, оставив его в трехмерном формате даже для одноканального изображения.
- ▶ Изменение параметра `padding = valid` уменьшит выходную размерность до 26x26. В случае его использования пиксели, выходящие за границы окна, не будут учитываться.
- ▶ При использовании `padding=same`, выходная размерность будет равна входной, а пиксели будут дополнены нулями.

<

Предыдущий урок

Следующий урок

>



Теперь мы готовы усложнить свёрточный слой и применить его к какой-нибудь цветной картинке.

Картинку будем описывать тензором размера $\mathbf{W} \times \mathbf{H} \times C_{in}$, где C_{in} — количество входных каналов (3 канала RGB).

Для операции свёртки будем теперь «вырезать» не квадратик, как раньше, а кубик, чтобы отличать, например, рыжих котов от чёрных. Фильтр теперь тоже тензор и его глубина также равна C_{in} .

Независимо от того, какой размерности у нас картинка и фильтр, на выходе всё равно будет скаляр, т.к. мы берем входной кусочек объёма и объёмный фильтр, перемножаем и складываем, получая скаляр. Т.е. применив свёртку к картинке, получим плоскую картинку.

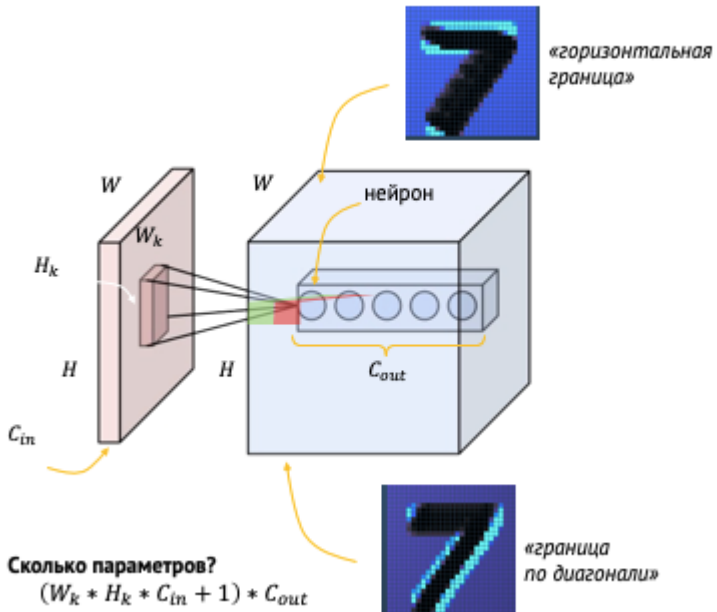
Мы потеряли достаточно много информации, ведь на вход подавалась трехмерная картинка, а на выходе получилась двумерная. Тут-то и становится понятно, что одного фильтра мало.

Как решить эту проблему?

Мы можем обучить много разных фильтров, все их применить, а результирующие картинки соединим в одну объёмную картинку на выходе. Каждый разрез этой объёмной выходной картинки будет хранить некую *карту фичей*, которую мы насчитаем.

Допустим, зелёный фильтр на картинке будет определять пиксели на горизонтальной границе, т.е. всегда, когда мы будем на горизонтальной границе изображения, этот нейрон будет принимать какое-то большое значение.

Красный же фильтр научим определять пиксели на границе объекта по диагонали. И теперь в одном пикселе входного изображения появляется целый вектор признаков, которые мы можем использовать для дальнейшего анализа. Этих признаков мы можем сделать сколько угодно. Это наш гиперпараметр C_{out} — количество выходных каналов в нашем объеме. Очевидно, что $C_{out} > 1$, а на практике может быть такое значение как 32 или 64.



Теперь у пикселя появляется глубина и мы переходим в новые координаты. Исходные координаты RGB были неудобны, т.к. в них ничего непонятно было о картинке, а теперь мы можем перейти, например, в 32 признака, каждый из которых решает свою определённую подзадачу. Мы сможем понять, на какой части картинки мы изображены: на горизонтальной или на диагональной.

Следующие фильтры, которые будут применяться, смогут учитывать все эти признаки одновременно и научатся решать более сложную задачу.

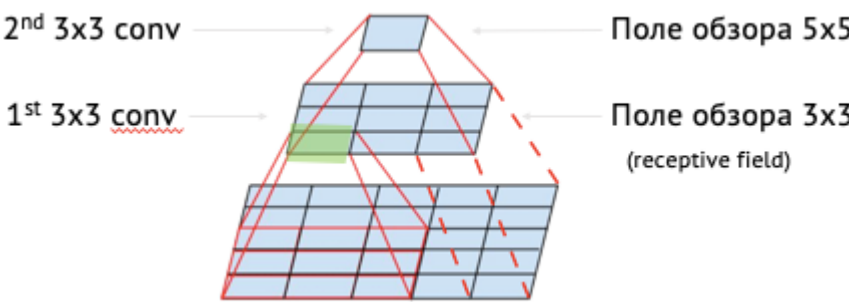
Вновь подчеркнём, как мало параметров мы использовали. Если наш фильтр имеет размеры $W_k \times H_k \times C_{in}$, то для того чтобы научить C_{out} таких фильтров, необходимо всего лишь $(W_k * H_k * C_{in} + 1) * C_{out}$ параметров.

Да, и одного свёрточного слоя мало!

Допустим, на картинке 300 × 300 изображен кот, а наш первый свеёрточный слой посмотрел на кусочки изображения 3 × 3. Эти кусочки слишком маленькие. Мы не можем из них собрать кота.

Давайте посмотрим, что произойдет с нашей картинкой и преобразованиями в нейросети, когда мы к первому результату нашей свёртки применим ещё один свёрточный слой.

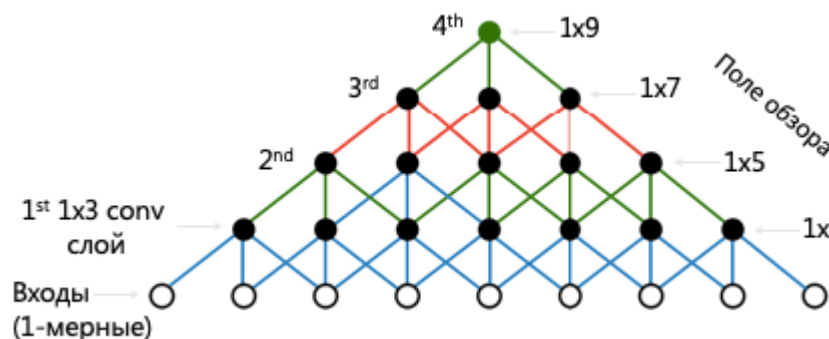
На примере ниже — вход размера 5 × 5. Мы применили свёртку размера 3 × 3 и получили 9 различных значений. Заметьте, что на первом свёрточном слое каждый нейрон (например, зелёный, выделенный на картинке) смотрит на кусочек изображения размером 3 × 3. Но если к результату первого свёрточного слоя ещё раз применить свёртку с какими-то другими значениями в фильтре того же размера 3 × 3, то любой нейрон на выходе второго слоя свёртки уже смотрит эффективно на кусочек картинки размером 5 × 5, т.к. все нейроны, с которыми слой работает, смотрят на целый кусочек размера 3 × 3. Если провести пунктирные линии, то видно, что у нашего нейрона «повысилось поле обзора» и он смотрит на больший кусочек изображения. То есть, создавая больше слоёв, мы имеем возможность определить котика всё большего и большего размера.



Что делать, если у нас котик занимает весь размер картинки (300 × 300)?

Посчитаем, сколько нам потребуется слоёв, чтобы распознать такого котика.

Рассмотрим это на простом одномерном примере. Допустим, у нас на входе одномерный вектор из 9 нейронов. Посмотрим, как будет выглядеть результат первой свёртки размером 3 × 1.



Каждый нейрон на первом свёрточном слое смотрит на 3 нейрона нашего изображения. Если мы применим ещё один свёрточный слой, то нейроны второго слоя будут смотреть уже на 5 пикселей нашего изображения. Если так продолжать делать, то можно вывести формулу. Видно, что наше поле обзора прирастает линейно по количеству слоёв. Значит, чтобы посмотреть на котика размером 300 × 300, нам понадобится 150 слоёв, а это очень много. Возникает проблема, потому что столько слоёв будут очень долго просчитываться. Чуть позже мы научимся её решать.

Практика. Реализация простой сверточной сети

В этом уроке мы изучим, как работают сверточные слои в нейронных сетях, как количество фильтров и слоев влияют на количество параметров в сети, как это влияет на процесс обучения и эффективность модели.

Обратите внимание, мы используем **ту же тетрадь, что и в прошлом уроке практики.**



← Предыдущий урок

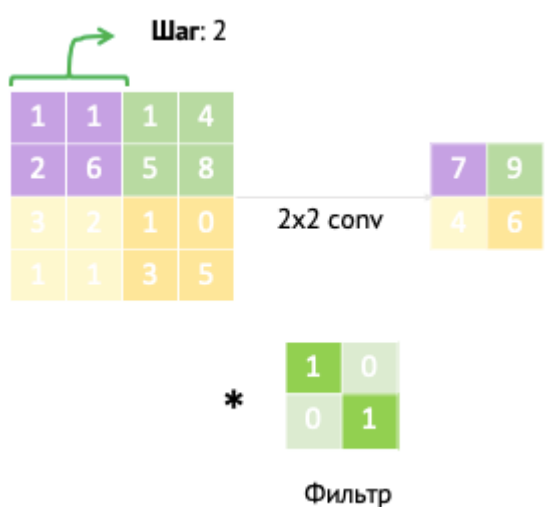
Следующий урок →



Нужно растить поле обзора быстрее!

Самый простой способ увеличить поле обзора наших нейронов — увеличить шаг свёртки.

Возьмём шаг нашего скользящего окна равным 2. Тогда наша картинка станет меньше вдвое (т.е. если наша картинка была размером 300 × 300, то после свертки с шагом 2 она стала 150 × 150). При добавлении новых свёрточных слоев наша картинка каждый раз будет уменьшаться вдвое, а значит, чтобы найти кота размером на всю картинку нам понадобится всего 9 слоев (а не 150 как раньше).

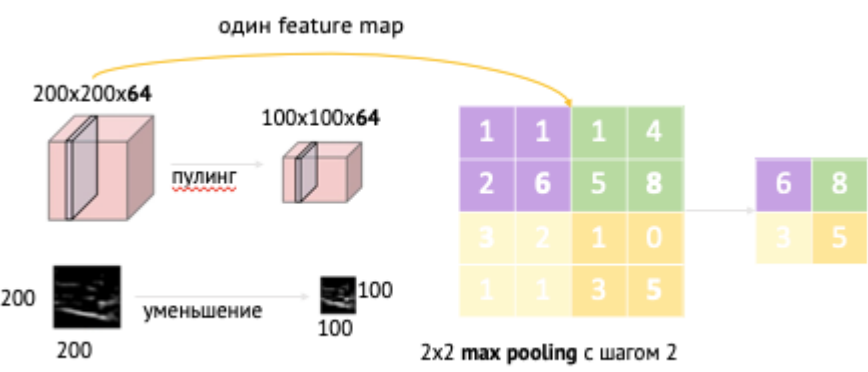


Вспомним хорошее свойство нашего классификатора: после применения свёртки мы брали максимум. Когда мы берем свёртку, мы считаем пространственную корреляцию нашего кусочка изображения с тем шаблоном, который мы хотим на нём найти. Там, где корреляция большая, мы нашли что-то похожее на наш шаблон, а затем мы брали максимум и понимали, что наша палочка находится под разным углом.

В нашем свёрточном слое мы нигде не использовали функцию максимума. Но нужно обязательно её использовать, чтобы была инвариантность к сдвигу изображения. В этом нам поможет *пулинг слой*.

Как работает пулинг слой?

На входе есть картинка (4 × 4) и есть окно фиксированного размера (2 × 2). Если мы проходим этим окном по картинке с шагом 2, то у окна есть всего 4 возможных положения. Теперь *пулинг слой* применяет некоторую операцию (мы возьмем максимум). В оранжевом окошке максимум 6 и поэтому на выходе получаем 6, в фиолетовом 8 и т.д.



Пулинг слой комбинирует в себе два плюса:

- Во-первых, он применяет максимум, а значит, имеет небольшую инвариантность к сдвигу.
- Во-вторых, *пулинг слой* обычно применяют с шагом 2, а это значит, что наша картинка уменьшается в 2 раза и поле обзора растёт быстрее.

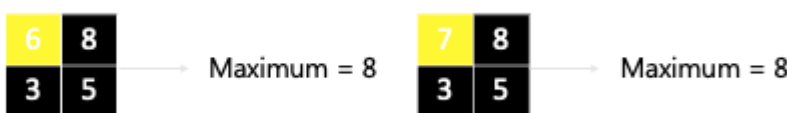
Пулинг с максимумом применяется к различным *feature map* независимо от объёма, который мы получили: он работает на входе и на выходе с плоскими картинками и не меняет количество фильтров.

Пулинг с максимумом теряет детали изображения (на картинке выше показан результат свертки 200 × 200, к которому применяют *пулинг* и в итоге получают картинку 100 × 100).

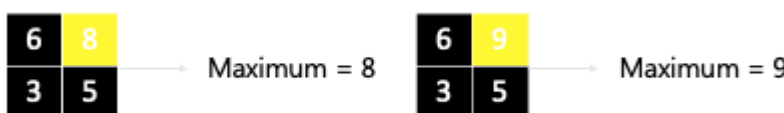
Как считать градиент для пулинга

Возникает ещё один важный вопрос: как считать градиент для *пулинга*, если операция максимума не дифференцируема?

Очевидно, что результат *макс-пулинга* не изменится, если мы немного подвинем входные данные, не являющиеся максимальным. Т.е. при варьировании этих не максимальных элементов, результат *макс-пулинга* не меняется, а значит, производная по этим элементам равна 0.



При изменении максимального элемента максимум сразу меняется и производная по этому элементу равна 1.



Таким образом мы можем посчитать градиент *пулинга*.

Подведем итоги

- Пулинг* применяется с БОЛЬШИМ *stride* и обеспечивает более быстрое увеличение поля обзора (экспоненциальный рост против линейного).
- Взятие максимума вместо обучаемой свёртки экономит вычисления и добавляет инвариантность к сдвигу.



9/13 Практика. Добавление Pooling-слоя

текущий урок

Практика. Добавление Pooling-слоя

В этом уроке мы попробуем добавить в нашу сеть pooling-слой. Pooling-слои могут быть добавлены для уменьшения размерности карты фичей и извлечения наиболее значимых признаков.

Обратите внимание, мы используем **ту же тетрадь**, что и в прошлом уроке практики.



◀ Предыдущий урок

Следующий урок ▶

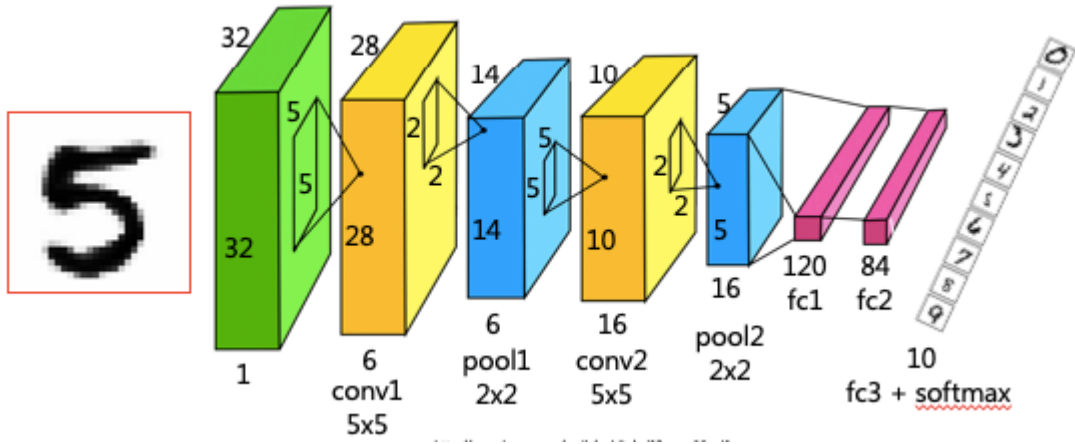
SKILLFACTORY

Соберем это все в свёрточную сеть

Теперь мы готовы собрать нашу первую свеёрточную сеть.

Для примера рассмотрим архитектуру **LeNet**, придуманную в 1998 году. Она применялась для задачи MNIST — задачи распознавания рукописных цифр.

На вход принималась чёрно-белая картинка размера 32 × 32 × 1. Сначала предлагается применить свёртку с фильтром размером 5 × 5 и обучить 6 таких фильтров, чтобы на выходе получить объем 28 × 28 × 6 (32 превратилось в 28 потому, что мы не использовали *padding*). Далее предлагается использовать *пулинг слой*, чтобы уменьшить картинку в 2 раза, т.о. получив размеры 14 × 14 × 6.



Количество фильтров при этом не поменяется, т.к. *пулинг* работает с каждым *feature map* независимо.

Потом применяем ещё один свёрточный слой с фильтром 5 × 5, чтобы увеличить поле обзора, и получаем размеры 10 × 10 × 16. Фильтров, которые обучаются, становится уже больше. Это делается для того, чтобы не терять информацию.

Вновь используем *пулинг слой*, который сделает нашу картинку размером 5 × 5 × 16. Полученный объём предлагается вытянуть в вектор и к этому вектору применить 2 полносвязных слоя, т.е. предполагается, что в этом последнем *пулинг слое* в 16 признаках уже закодирована нужная информация, которая поможет нам воссоздать цифру.

После применения двух полносвязных слоёв нужен выходной слой, на котором будет 10 выходов, т.к. у нас 10 цифр, и будет применена функция *softmax*, которая превратит любые выходы в правильное распределение вероятностей.

Если взять 60 000 примеров, где у нас есть картинка на входе и для неё известен класс, собрать эту архитектуру и оставить *backpropagation* на пару минут, то он выучит все веса на всех слоях, которые решают эту задачу практически идеально.

Как работает функция softmax?

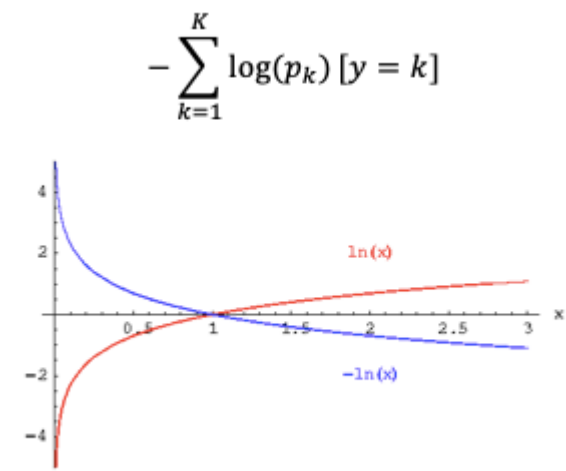
Допустим, мы посчитали 10 выходов. Каждый из них заменим на экспоненту в степени этого выхода и пронормируем все полученные экспоненты, чтобы в сумме они давали 1. Т.к. экспонента неотрицательна, а их сумма равна 1, то получим правильное распределение вероятностей.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$$

Кросс-энтропия

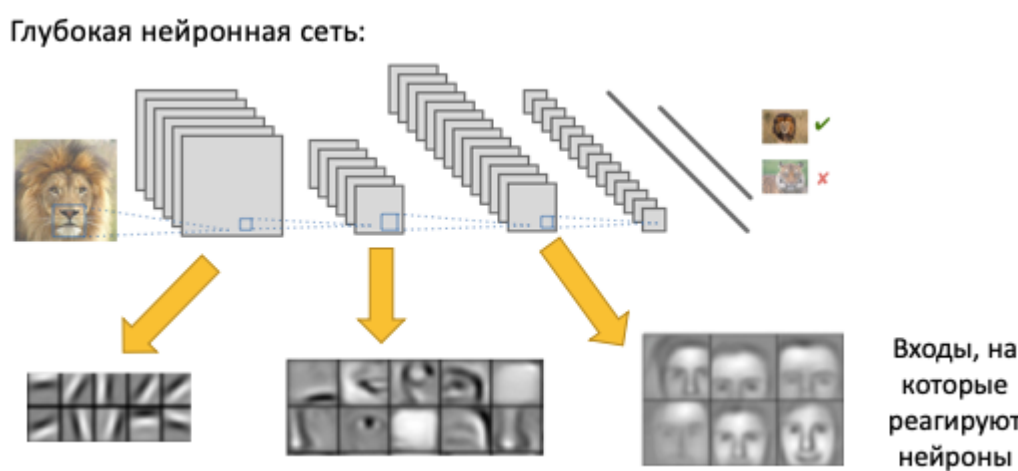
Чтобы запустить *backpropagation*, нужно знать какую функцию потерь использовать. Для классификации на *K* классов обычно используют функцию потерь, которую называют **кросс-энтропия** (cross-entropy).

Для каждого примера мы идем по всем возможным классам и, если пример принадлежит этому классу, то мы в потери записываем логарифм со знаком «минус» от предсказанной вероятности этого класса.



По графику функции -ln(x) видно, что если аргумент этой функции близок к 1, то потери близки к 0, а если аргумент близок к 0, то потери уходят в бесконечность. Это означает, что в потерях на правильном классе хочется видеть вероятность как можно более близкую к 1. Это и будет решать нашу задачу классификации.

Если обучить нашу нейросеть и посмотреть на каждом слое изображения, на какие кусочки реагируют (имеют большую активацию) наши нейроны, то заметим, что на первом слое нейроны реагируют на простые *паттерны* на изображении (например, они могут находить перепады яркости под разными углами). Второй свёрточный слой уже может распознавать что-то более сложное (например, кусочек глаза) и т.д. Таким образом, нейросеть учит иерархические шаблоны.



Практика. Добавление Flatten и Dense слоев. Обучение модели

В этом уроке мы дособираем и обучим нейронную сеть для задачи распознавания одежды по изображениям из датасета.

Обратите внимание, мы используем **ту же тетрадь, что и в прошлых уроках практики.**



← Предыдущий урок

Следующий урок →

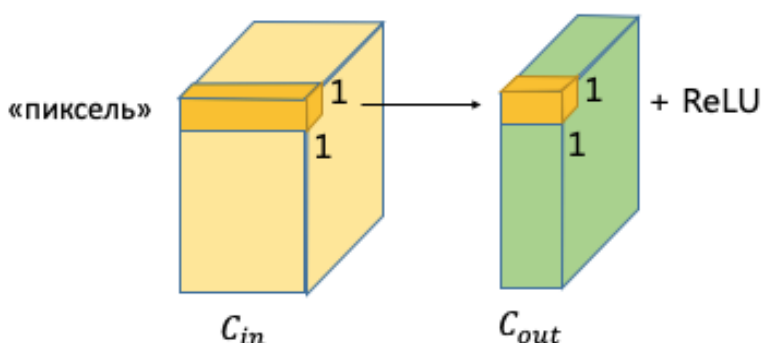


Теперь познакомимся с архитектурой Inception V3, предложенной также в 2015 году и отличающейся от AlexNet. В этой сети значительно меньше параметров (всего 25 миллионов). Эта сеть состоит из оригинальных *inception-блоков* (их рассмотрим чуть позже) и дополнительных трюков таких как *батч нормализация*, аугментация и *RMSProp* (алгоритм оптимизации).

Как устроен inception-блок?

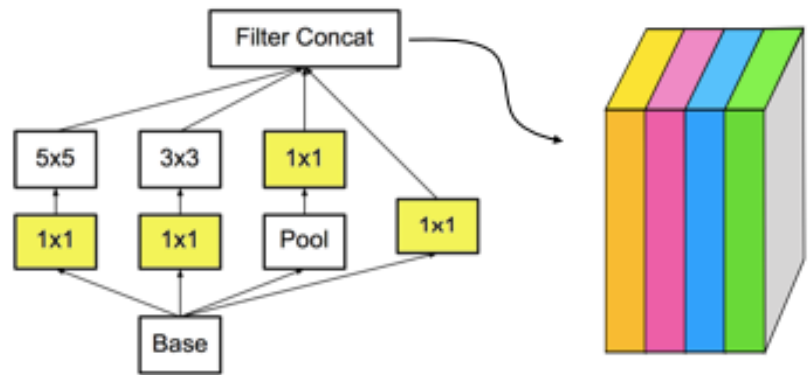
Первое с чем нам нужно познакомиться — свёртка 1×1 .

Пусть у нас есть входной объём (светло-оранжевый на картинке). И если у нас есть окошко 1×1 — это означает, что мы берем глубокий пиксель из синего объёма и хотим получить другой пиксель другой глубины.



Т.е. мы будем искать новые пиксели как линейные комбинации входных. Такое линейное преобразование очень похоже на уменьшение размерности.

Объясняя простым языком, мы даем нейросети возможность обучать много фильтров, а затем похожие фильтры схлопываются в один линейным преобразованием.

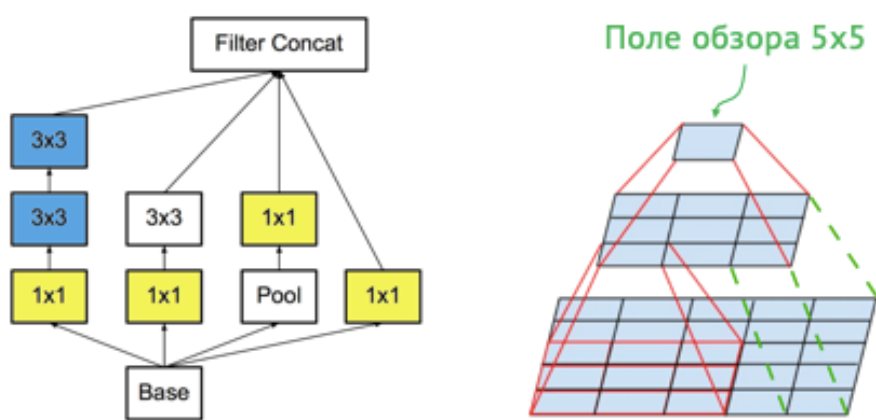


Как устроен inception-блок внутри?

На входе нам поступает некоторый объём и к нему применяют 4 ветки вычисления: первая ветка применяет свёртку 1×1 , чтобы уменьшить глубину этого объёма; дальше применяется свёртка 5×5 с таким *padding*, чтобы на выходе высота и ширина изображения остались неизменными; на второй ветке применяем свёртку 1×1 , а затем 3×3 ; на третьей ветке применяем пулинг с шагом 1, а затем свёртку 1×1 ; четвертая ветка просто делает свёртку 1×1 .

Отметим, что можно так подобрать *padding* и *stride*, что измерения W и H выходных объёмов всех веток будут одинаковыми, а глубина будет различаться. Это означает, что их можно будет склеить по глубине. В итоге получим один большой объём.

Можно заменить свёртки 5×5 двумя свёртками 3×3

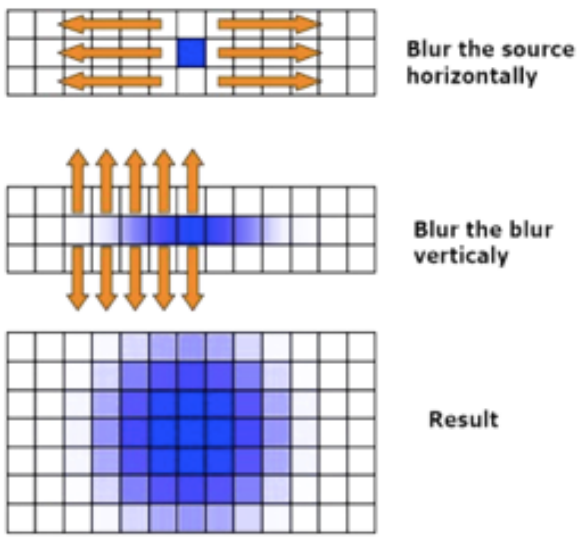


Количество параметров, необходимых для такой операции, станет меньше.

Сепарабельные фильтры

Существует фильтр размытия по Гауссу — гауссиана, т.е. мы возьмем кусочек изображения, взвесим все пиксели с весами, которые даёт нам гауссиана и получим размытие по гауссу. Это размытие можно «дешево» сделать одномерными свёртками.

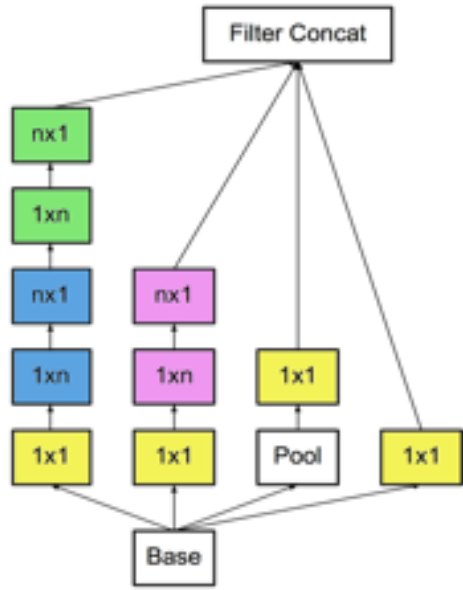
Например, пусть у нас есть синий пиксель. Сначала сделаем размытие одномерным гауссовским ядром по горизонтали:



К этому результату можно применить свёртку по вертикали и мы получим такой же результат, как если бы мы применяли двумерную свёртку к нашему пикселю. Если посчитать количество операций при размытии по горизонтали, а затем по вертикали, то количество операций окажется в разы меньше.

Такие фильтры, которые позволяют заменить себя на последовательность одномерных, называются **сепарабельными**.

Дадим нейросети возможность учить такие фильтры, заменим в нашем *inception-блоке* свёртки 3×3 на сепарабельные свёртки 1×3 и 3×1 .



Таким образом, мы придумали очень эффективный *inception-блок*, который использует мало параметров и дает лучшее качество, чем набор свёрток 3×3 .



В этом модуле мы хотим рассказать вам о:

- ▶ функциях активации;
- ▶ инициализации весов;
- ▶ batch-нормализации;
- ▶ dropout-регуляризации;
- ▶ градиентном спуске;
- ▶ стохастическом градиентном спуске;
- ▶ матричных операциях.



Функция активации нейрона определяет выходной сигнал, который, в свою очередь, определяется входным сигналом или набором входных сигналов. Функцию активации используют, чтобы получить выходные данные узла.



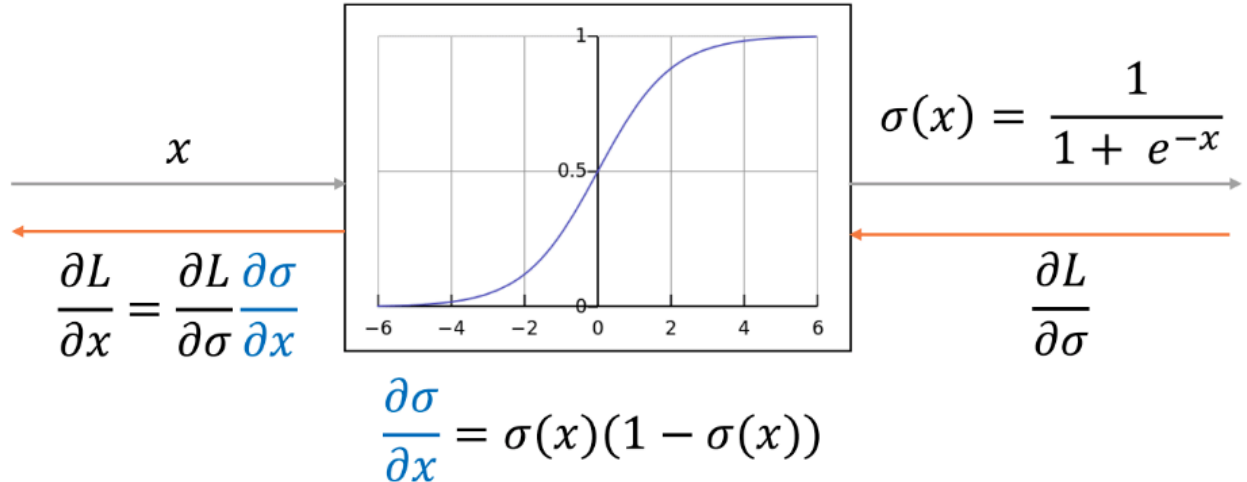
Функции активации делятся на два типа:

1. Линейные функции активации;
2. Нелинейные функции активации.

Сигмода (Sigmoid) — возрастающая нелинейная функция, имеющая форму буквы «S». В нейронных сетях она используется, поскольку позволяет усиливать слабые сигналы.

Проблемы Sigmoid-активации (да, ей тоже нелегко живется):

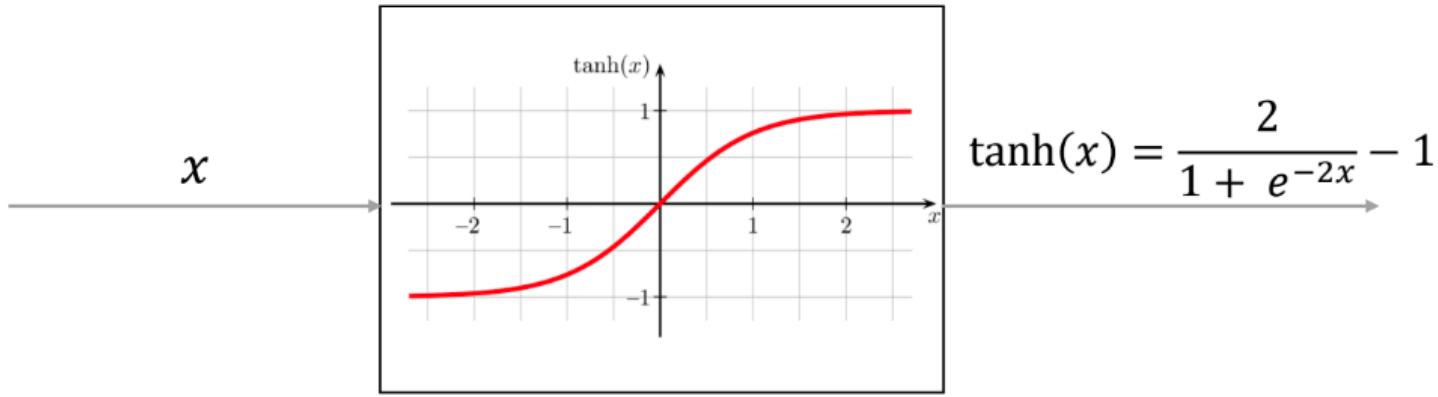
- ▶ Нейроны с сигмодой могут насыщаться и приводить к угасающим градиентам;
- ▶ Не центрированы в нуле;
- ▶ Дорого вычислять.



Перейдем к функции Tanh. Здесь все просто: она похожа на сигмоиду, но у функции Tanh есть преимущества: отрицательные входные данные будут отображаться строго отрицательными, а нулевые входные данные будут отображаться вблизи нуля.

Характеристики Tanh активации:

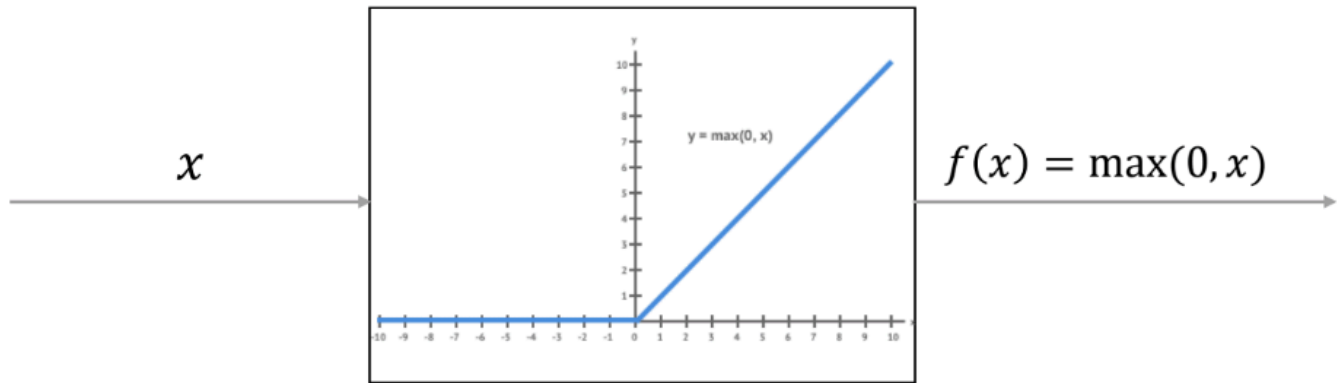
- ▶ Центрирована в нуле;
- ▶ Но все ещё как сигмоида.



ReLU (rectified linear unit)

Характеристики ReLU активации:

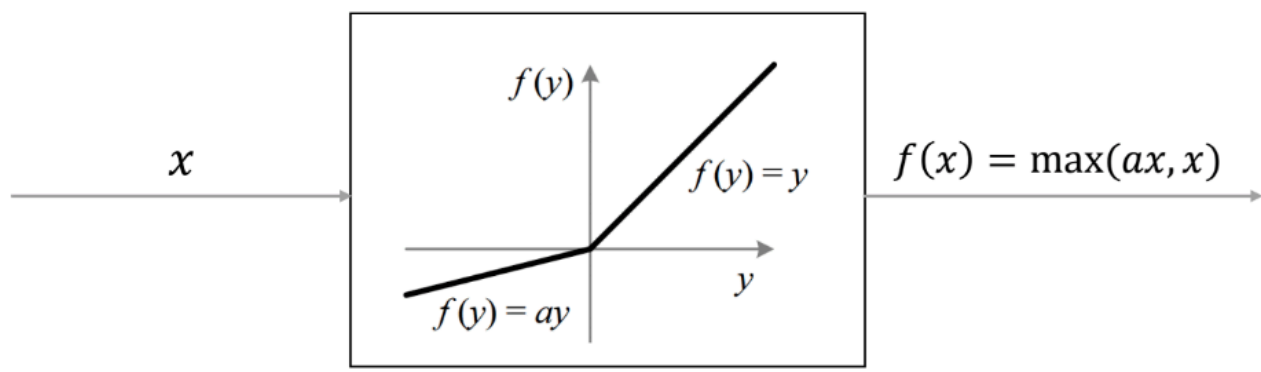
- ▶ Быстро считается;
- ▶ Градиенты не угасают при x > 0;
- ▶ Не центрирована в нуле;
- ▶ Если не было активации, не будет обновления.



Leaky ReLU — это попытка решить проблему выхода ReLU из строя.

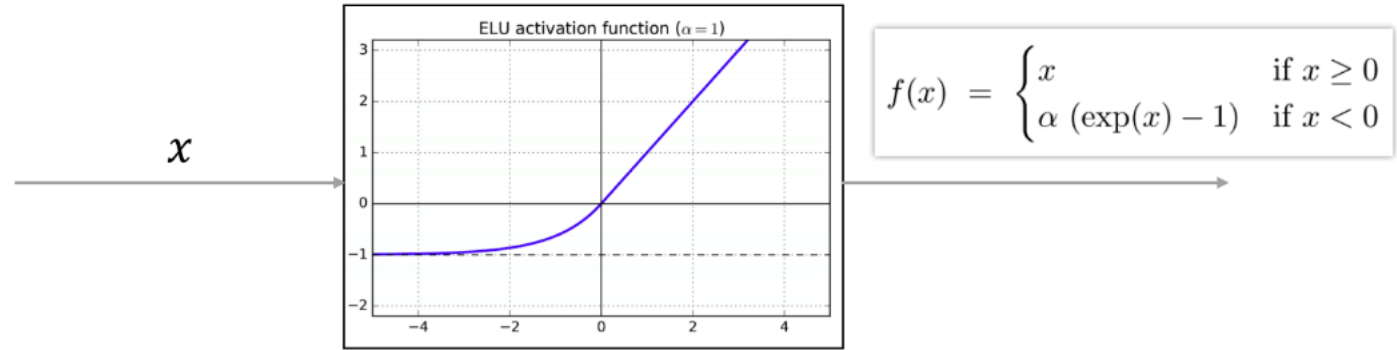
Характеристики Leaky ReLU-активации:

- ▶ Всегда будут обновления;
- ▶ Примерно центрирована в нуле;
- ▶ a ≠ 1.



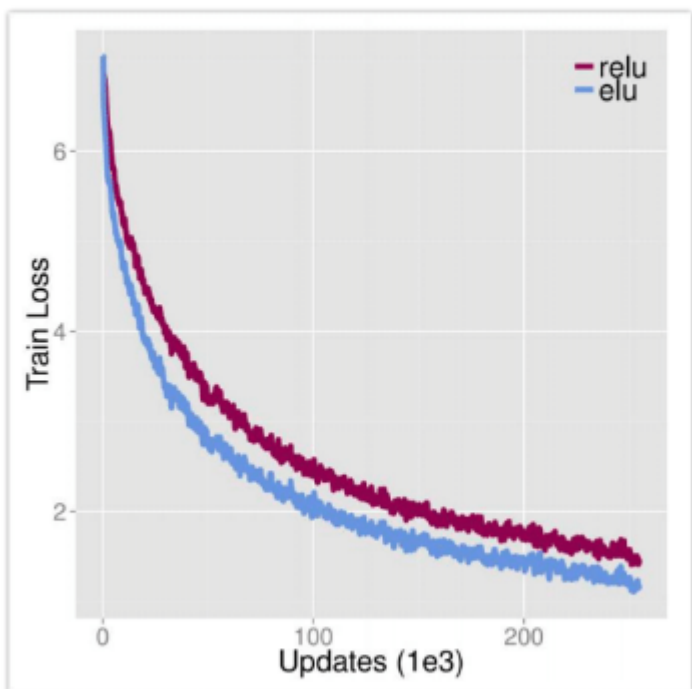
Функция активации ELU (Exponential Linear Unit) по результатам исследований быстрее сводит к нулю и даёт более точные результаты.

В отрицательной части аргументов использует экспоненту.



Характеристики ELU:

- ▶ Примерно центрирована в нуле;
- ▶ Сходимость быстрее ReLU.



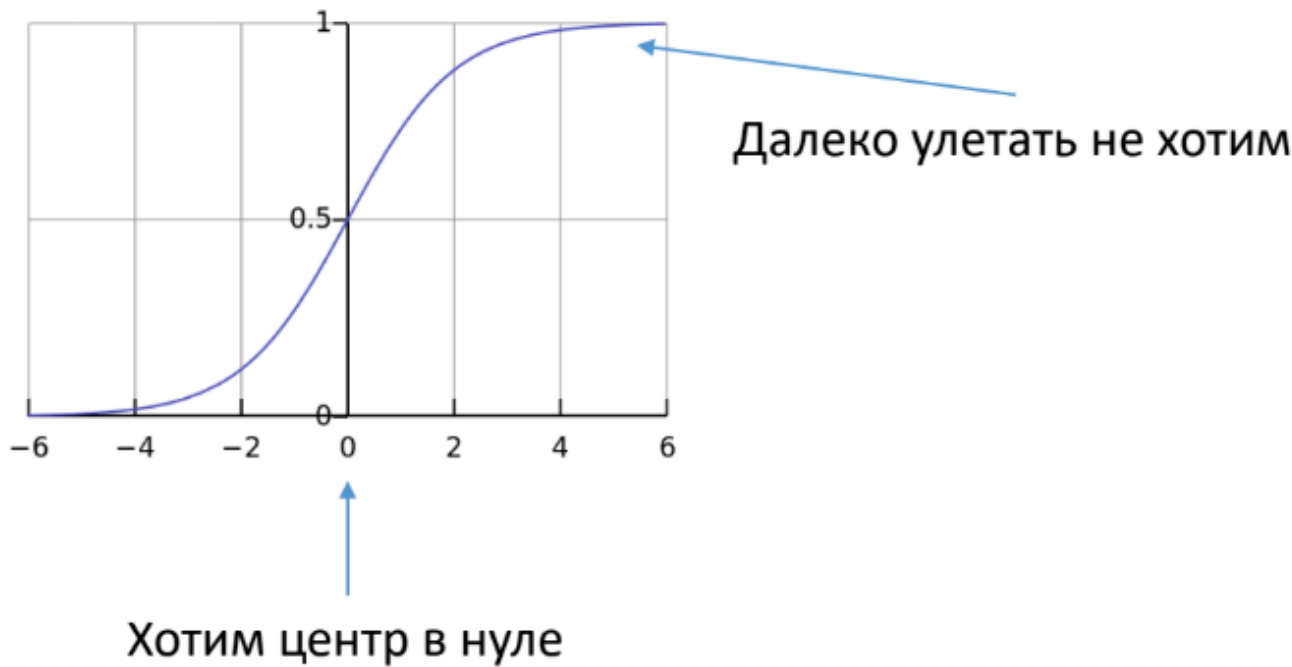


Нейронные сети нельзя инициализировать нулевыми весами, так как в этом случае появляется эффект симметрии и, как следствие, сеть становится менее мощной.

НЕЙРОН И ДИСПЕРСИЯ ДО АКТИВАЦИИ

Если у каждого входа нейросети среднее равно 0, и мы генерируем веса независимо от входов, то это нам гарантирует, что среднее нейрона до активации тоже будет равно 0.

Но дисперсия может расти, и это замедлит сходимость.



Дисперсия суммы некоррелированных величин — это сумма дисперсий, слагаемые некоррелированы, потому что веса, которые мы генерируем, независимы.

СХОДИМОСТЬ

Гиперболический тангенс для маленьких функций (около 0) похож на линейную функцию, то есть наши ранние выкладки примерно верны, и это означает, что, так как мы гарантируем одинаковую дисперсию на разных слоях нейросети, то выход будет распределён примерно одинаково после применения функции активации и, следовательно, градиенты будут в примерно одинаковом масштабе. Этот факт позволит ускорить сходимость.

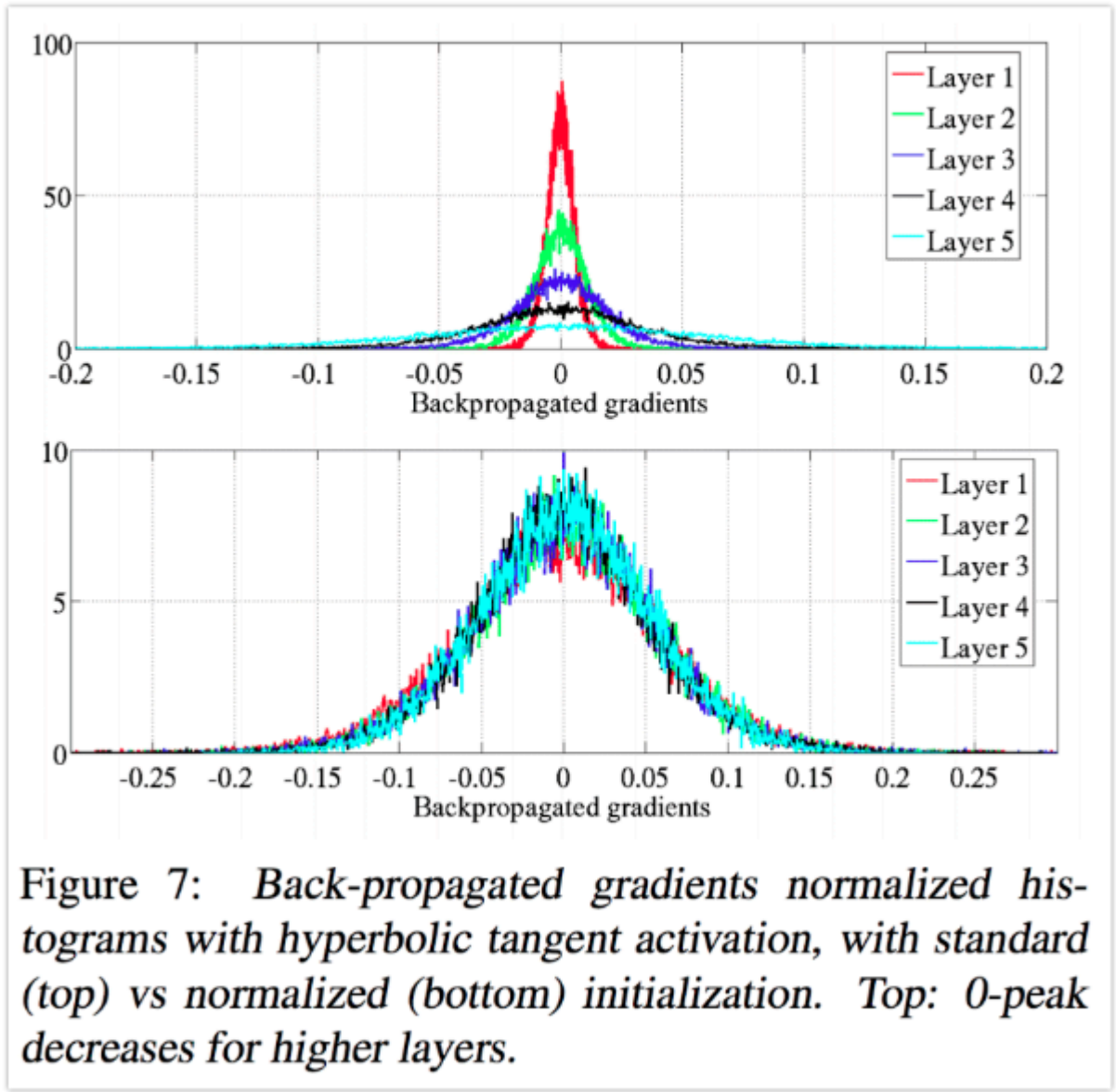


Figure 7: Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.

← Предыдущий урок

Следующий урок →



↓ Используемый в видео Notebook

Рассмотрим, как масштаб признаков в линейной регрессии влияет на сходимость градиентных методов.

Начнём с импорта библиотек

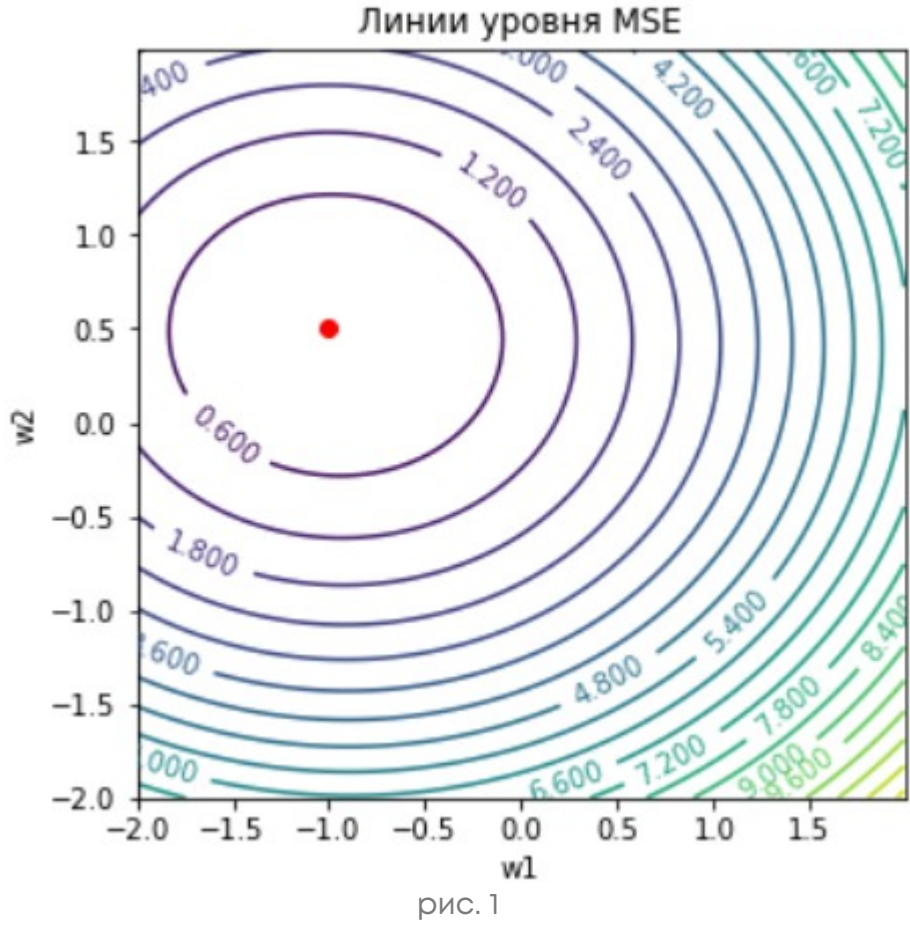
Здесь нам понадобится **TensorFlow** для визуализации процесса градиентного спуска и вспомогательные функции, которые будут отрисовывать анимацию.

Задача регрессии:

У нас есть два признака: x_1 , x_2 , на которых настроена линейная модель для предсказания y :

$$\text{Лосс на одном объекте: } l = (x_1 w_1 + x_2 w_2 - y)^2$$

Среднеквадратичное отклонение будем рисовать в пространстве параметров при помощи линий уровня. **Линия уровня** — это множество точек, на которых функция принимает одно и то же значение.



Глядя на линии уровня (рис. 1), мы можем представить, как устроена трёхмерная поверхность функции двух переменных w_1 и w_2 .

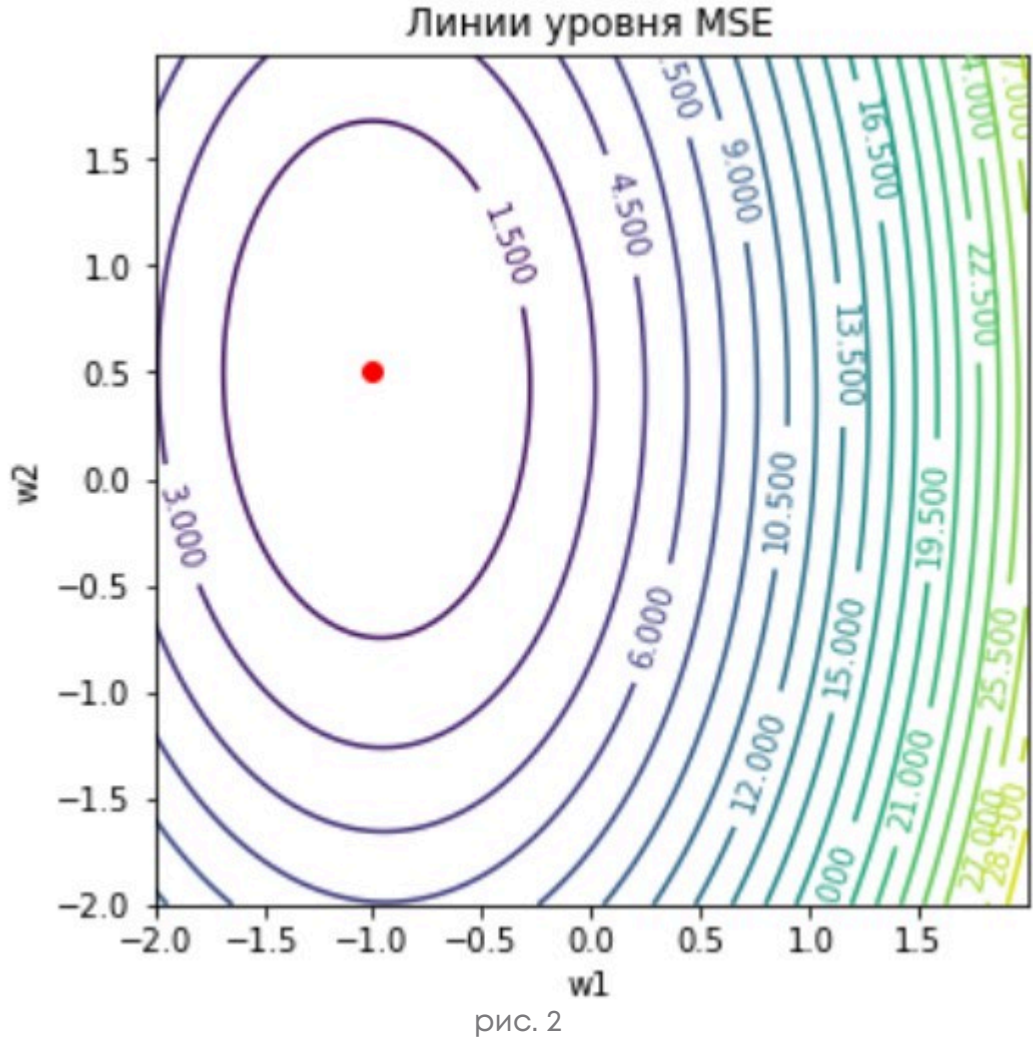
Красная точка на рисунке — это оптимальный набор параметров.

Рассмотрим, как выглядят матрицы W_1 и W_2 . Для расчёта функции потерь вытянем их в вектор:

```
Z = np.mean((x.dot(np.vstack([W1.ravel(), W2.ravel()]))) - y)**2, axis=0).reshape(W1.shape)
```

Рассмотрим, как изменятся линии уровня, если ось x умножить на коэффициент 2, например:

```
x[:, 0] *= 2 # признак x1 в x_scale раз больше (создает долины в лоссе)
```



Эллипс сузился по горизонтальной оси (рис. 2).

Если мы будем увеличивать масштаб x_1 , это будет сплющивать ось w_1 на графике линий уровня.

Если x_1 — большое число, то при небольшом изменении w_1 лосс изменится сильно.

Можно посчитать градиенты функций потерь по w_1 и w_2 :

$$\frac{dl}{dw_1} = 2(x_1 w_1 + x_2 w_2 - y) x_1$$
$$\frac{dl}{dw_2} = 2(x_1 w_1 + x_2 w_2 - y) x_2$$

Когда x_1 и x_2 будут иметь сильно разный масштаб, градиенты по параметрам тоже будут иметь сильно разный масштаб. И это именно то, что будет ломать градиентный спуск.

Далее разберём функцию, которая будет рисовать градиентный спуск, и также поэкспериментируем с линиями уровня.

← Предыдущий урок

Следующий урок →

4/11

Batch-нормализация

текущий урок



После инициализации и запуска **backpropagation** нет гарантий, что дисперсия не будет расти. Для нормирования выходов нейронов (до и после активации) можно использовать **Batch-нормализацию**.



Batch-нормализация пытается нормировать каждый выход нейрона таким образом, чтобы у него были нулевое среднее и фиксированная дисперсия.

Экспоненциальное сглаживание работает следующим образом: когда мы получаем среднее по батчу и Var по батчу, мы добавляем их в накопитель с некоторым весом, а старое значение в накопителе немного дискаунтируем.

Что делать с γ и β ?

γ и β нужны для масштабирования и сдвига активации: нейросети может быть полезно иметь, например, не единичную дисперсию для какого-то нейрона, а большую, что поможет ей сойтись лучше.

Линейная комбинация каких-то других входов легко дифференцируется, поэтому эти коэффициенты можно добавить в **backpropagation**, по ним будут считаться градиенты, и они будут автоматически подобраны.

←

Предыдущий урок

Следующий урок

→

5/11 Dropout-регуляризация

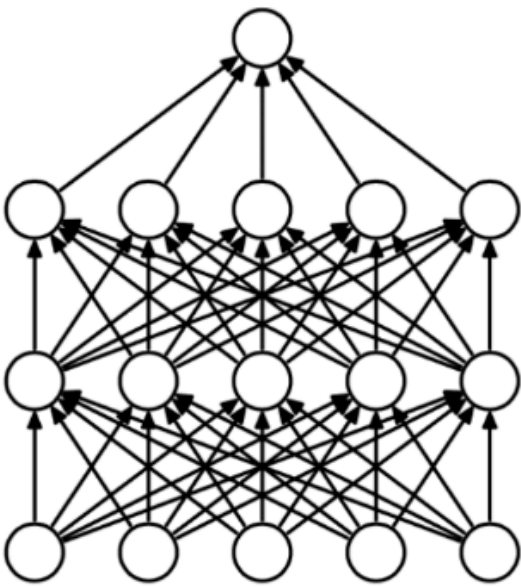
текущий урок

SKILLFACTORY

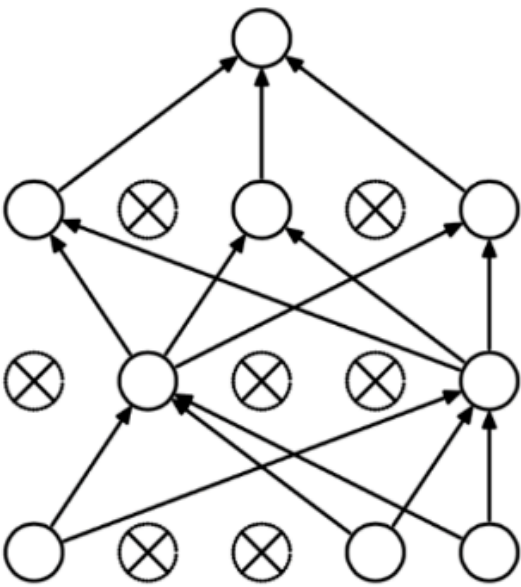
1x

0:01 / 3:17

Dropout-регуляризация на каждом шаге backpropagation семплирует сеть и с вероятностью p оставляет каждый нейрон, но с вероятностью $1 - p$ заменяет его на 0. Таким образом сеть подстраивается по частям.



(a) Standard Neural Net



(b) After applying dropout.

< Предыдущий урок

Следующий урок >

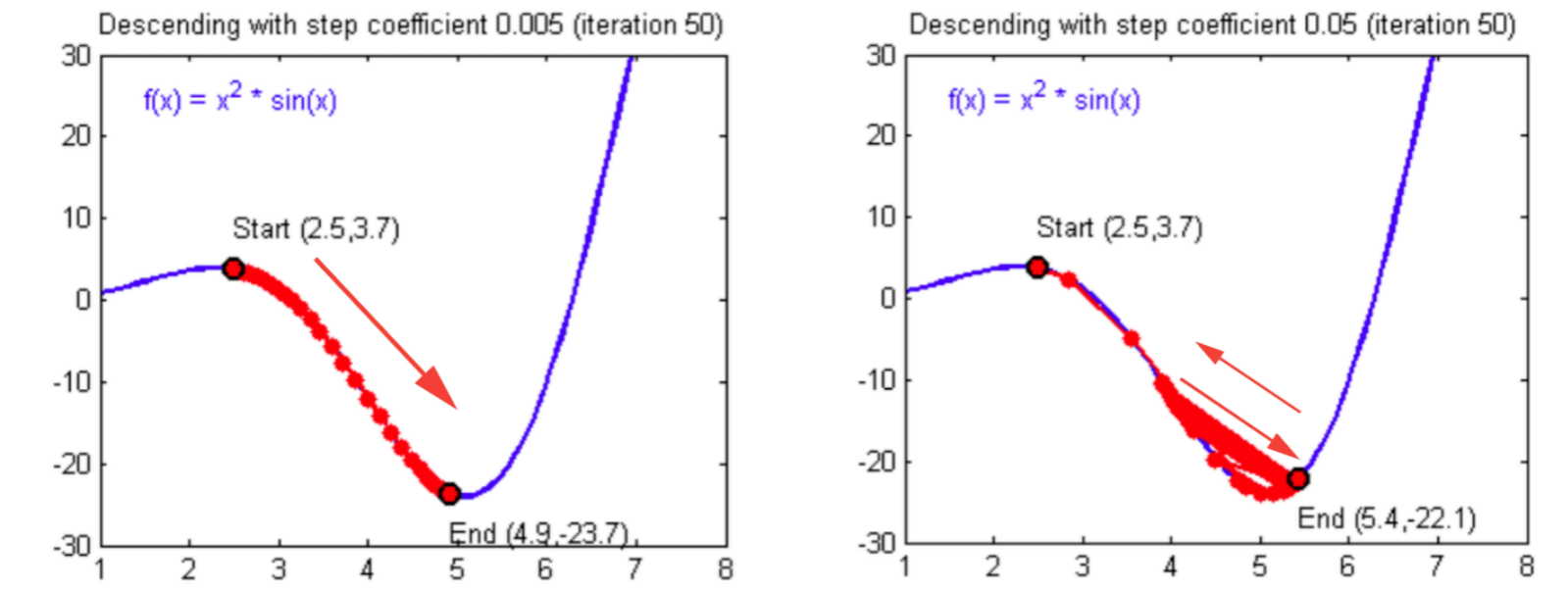
6/11 Производная функции

текущий урок



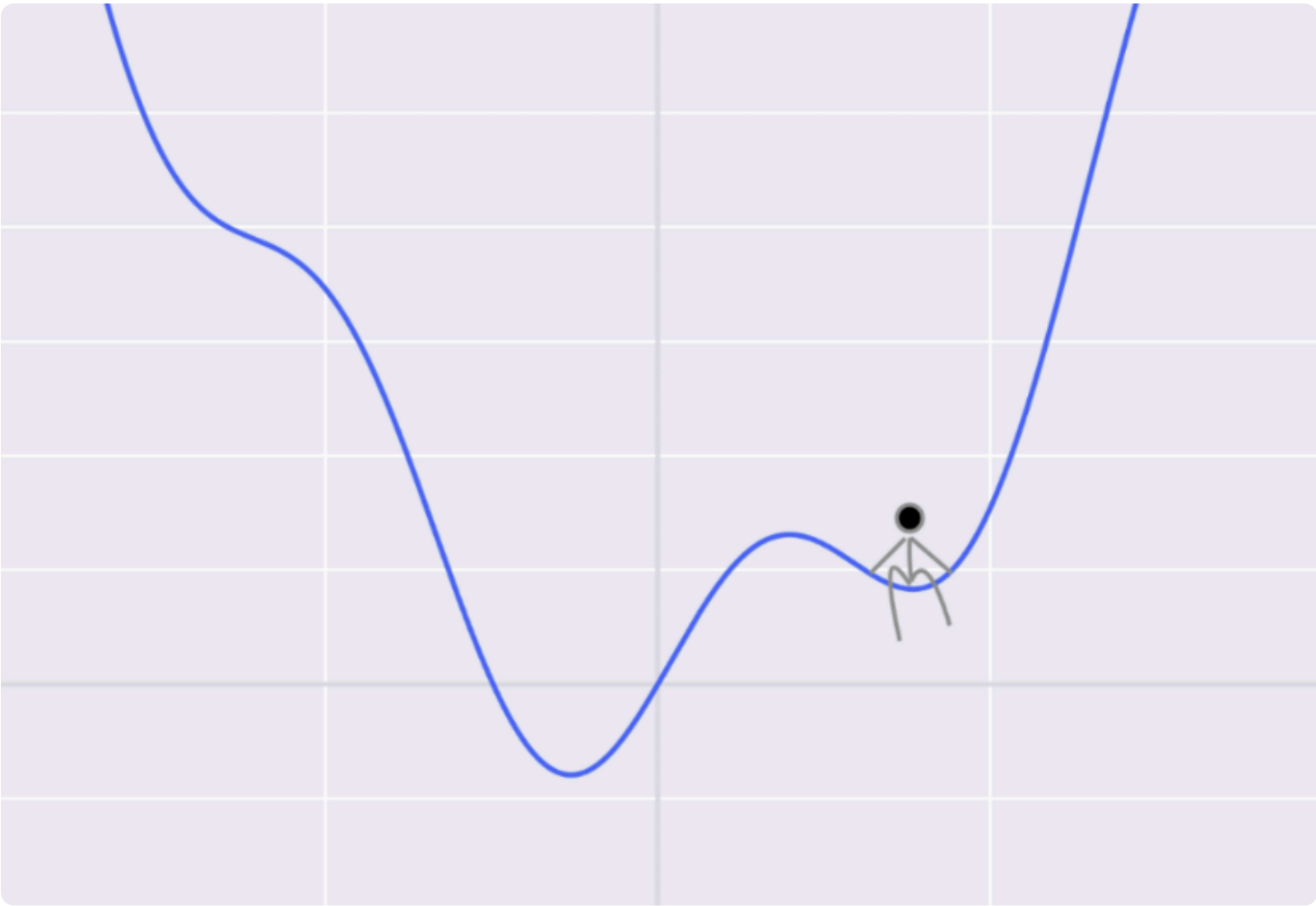
В зависимости от того, какую величину шага в градиентном спуске learning rate вы выберете (большую или маленькую), вариации градиентного спуска будут выглядеть по-разному:

- Если learning rate **слишком большой**, это означает, что сначала оптимизация была, но остановилась и качество становится хуже, мы выходим из оптимальной точки.
- Если learning rate **слишком маленький**, то обучение слишком медленное.
- Если learning rate **недостаточно большой**, то поначалу обучение идёт отлично, а потом мы застреваем и не двигаемся дальше.
- Если learning rate **оптимальный**, то мы обучаемся быстро вначале, и к концу мы выходим на оптимальную точку.



Почему стоит снижать learning rate с каждой итерацией? Если уменьшить learning rate, когда вы увидели, что обучение больше не происходит, то сеть ещё чуть-чуть доучится.

Проблема градиентного спуска — он застревает в локальных минимумах.



< Предыдущий урок

Следующий урок >



7/11

Стохастический градиентный спуск (SGD)



Стохастический градиентный спуск (SGD) — это итерационный метод оптимизации с дифференцируемой целевой функцией, а суть градиентного спуска — минимизировать функцию, делая небольшие шаги в сторону наискорейшего убывания функции.

Плюсы SGD:

- ▶ Позволяет быстрее делать шаги и быстрее сходиться;
- ▶ Траектория становится более шумной, что помогает выпрыгивать из локальных оптимумов.

RMSProp — адаптивный шаг. RMSProp — метод, в котором скорость обучения адаптируется для каждого из параметров.



Градиентные методы медленно сходятся, если градиенты по разным параметрам в разном масштабе. Как их привести в один масштаб? Если мы делали несколько маленьких шагов по какой-то переменной, то можно увеличивать шаг, чтобы не делать оптимизацию слишком медленной по некоторым переменным. То есть мы делим скорость обучения для веса на скользящее среднее значение после градиентов для этого веса.

Adam — Adaptive Moment Estimation. Adam — метод, в котором сочетаются инерция и адаптивность шага. Если перед вами стоит выбор вариации SGD, то выбирайте Adam.

<

Предыдущий урок

Следующий урок

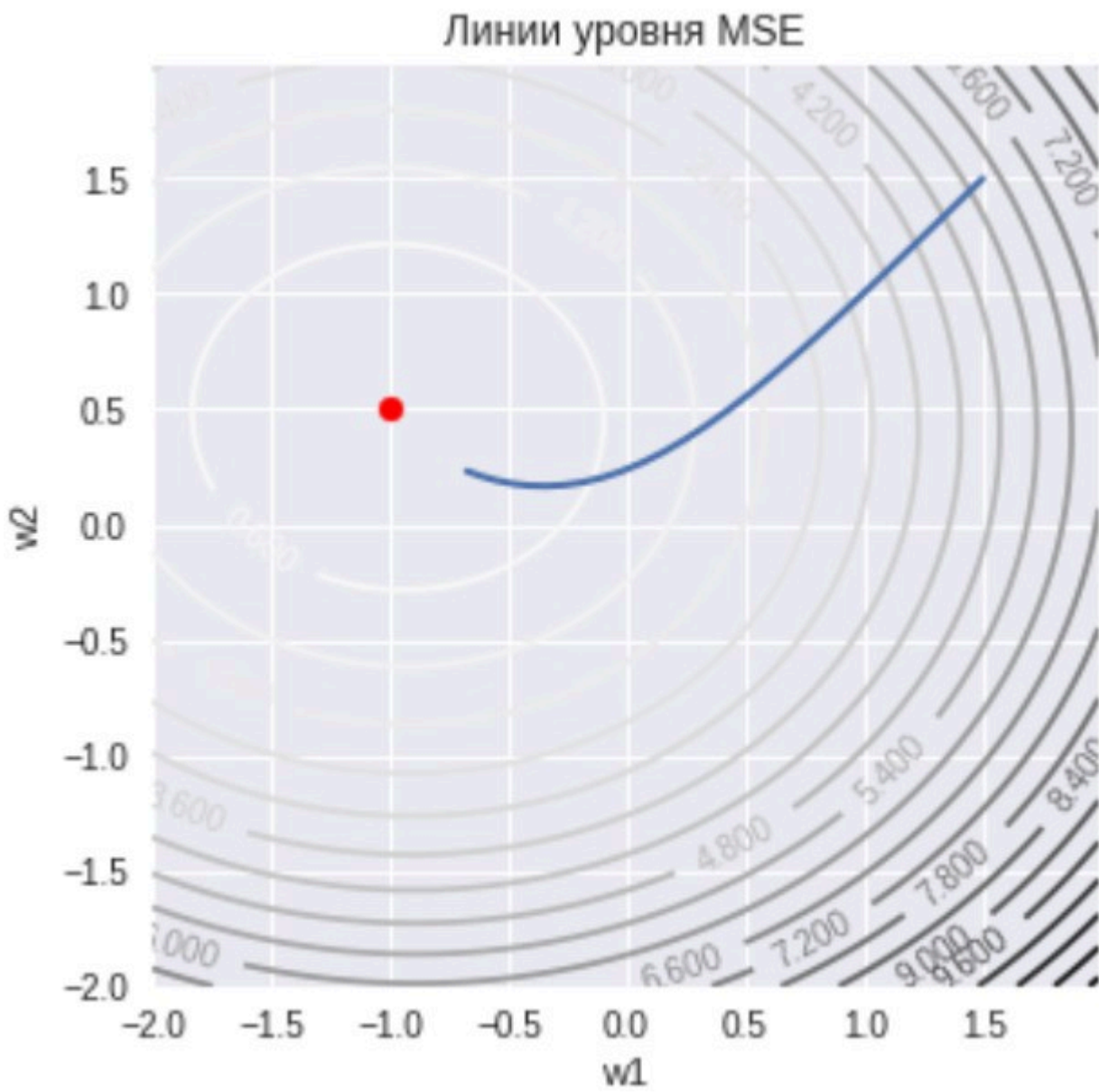
>



↓ Используемый в видео Notebook

В предыдущей задаче оптимизации MSE мы остановились на том, что при увеличении learning rate возникает осциллирование траектории.

Теперь рассмотрим на практике Adam. Запустим его на задаче, в которой признаки (а значит, и градиент) приблизительно одинакового масштаба.



Сплюснив вертикальную ось, заметим, что линии уровня станут сплюснутыми, а Adam сходится всё так же быстро.

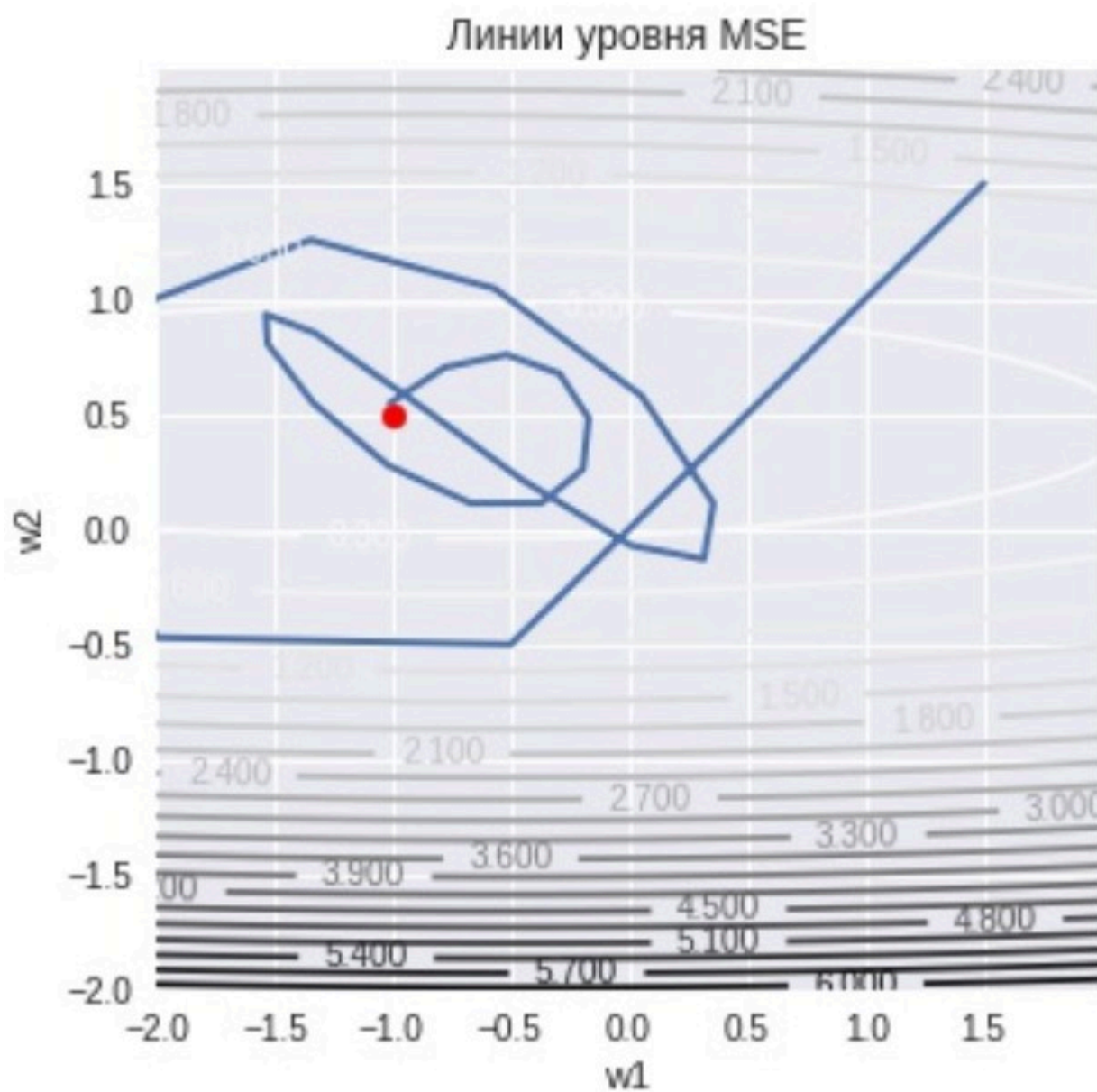
```
# узкие долины функции
plot_gd(x_scale=0.5, lr=0.1, steps=25, optimizer='adam')
```

Всё дело в адаптивном шаге, который нормирует градиенты и позволяет оптимально двигаться во всех направлениях. Давайте убедимся в этом, задавая ещё более узкие долины.

Теперь увеличим learning rate в 20 раз.

```
# ломает ли его шаг побольше?
# слишком большие шаги, закручиваемся вокруг решения из-за момента
plot_gd(x_scale=0.2, lr=2.0, steps=25, optimizer='adam')
```

Видим, что Adam шагает слишком далеко, и его траектория выглядит курьёзно.



Но, тем не менее, в итоге мы сошлись к самому хорошему решению.

Learning rate — это гиперпараметр, который нужно подбирать, и всё, конечно, зависит от того, как выглядит форма Loss-функции.

< Предыдущий урок

Следующий урок >

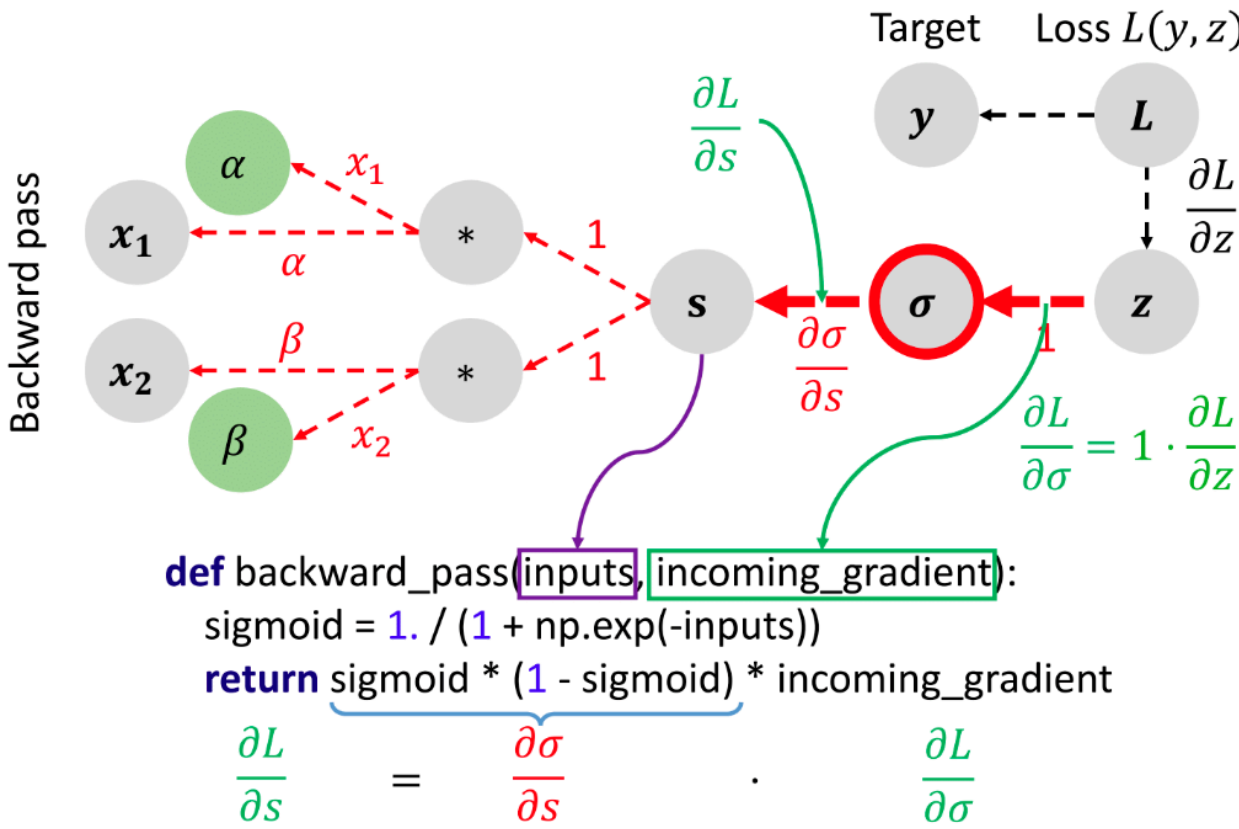


Интерфейс прямого прохода

Для прямого прохода интерфейс очень простой — это функция, которая принимает вход и генерирует выход, то есть один вход и один выход.

Обратный проход производится по графу из производных, то есть, когда мы идём по нему в обратную сторону, мы рассчитываем с помощью цепного правила производные по всем параметрам, которые есть в сети.

Чтобы реализовать Backward pass для новой вершины, нужно реализовать функцию, у которой есть входы — то, что использовалось как вход сигмоиды во время прямого прохода. Так как необходимо считать производную в какой-то точке, сначала backpropagation делает прямой проход, запоминает все значения аргументов, а потом использует их во время обратного прохода, чтобы рассчитать производные функций в нужных точках.



Полносвязный слой как произведение матриц

Матричные операции используются часто, они реализованы быстро. Существуют такие пакеты, как CPU (BLAS) и GPU (cuBLAS), которые производят численные операции очень быстро, используя векторные операции процессора.

Обратный проход

Для обратного шага необходимо посчитать производную наших потерь (обычно это скалярное значение) по каждому весу, который мы использовали.

Быстрая реализация в NumPy

Производит матричные операции очень быстро, не на Python, и NumPy можно настроить таким образом, чтобы он использовал те же инструкции, что заложены в пакетах BLAS.



Список литературы:

- ▶ Deep Learning // MIT Press [Электронный ресурс]. URL: <https://www.deeplearningbook.org/>
- ▶ Dive into Deep Learning // D2L.ai [Электронный ресурс]. URL: <https://d2l.ai/>
- ▶ Deep Learning with PyTorch // Manning Publications [Электронный ресурс]. URL: <https://www.manning.com/books/deep-learning-with-pytorch>
- ▶ Programming PyTorch for Deep Learning // O'Reilly Media [Электронный ресурс]. URL: <https://www.oreilly.com/library/view/programming-pytorch-for/9781492045342/>
- ▶ Deep Learning with Python // Manning Publications [Электронный ресурс]. URL: <https://www.manning.com/books/deep-learning-with-python-second-edition>
- ▶ Natural Language Processing with Transformers // O'Reilly Media [Электронный ресурс]. URL: <https://www.oreilly.com/library/view/natural-language-processing/9781098103231/>
- ▶ Deep Learning for Coders with fastai and PyTorch // O'Reilly Media [Электронный ресурс]. URL: <https://www.oreilly.com/library/view/deep-learning-for/9781492045519/>
- ▶ Mathematics for Machine Learning // Cambridge University Press [Электронный ресурс]. URL: <https://mml-book.github.io/>
- ▶ Optimization for Machine Learning // MIT Press [Электронный ресурс]. URL: <https://mitpress.mit.edu/9780262016469/optimization-for-machine-learning/>
- ▶ PyTorch Documentation // PyTorch Foundation [Электронный ресурс]. URL: <https://pytorch.org/docs/>
- ▶ TensorFlow Documentation // Google [Электронный ресурс]. URL: <https://www.tensorflow.org/learn>
- ▶ Hugging Face Transformers Documentation // Hugging Face [Электронный ресурс]. URL: <https://huggingface.co/docs/transformers>

◀

Предыдущий урок

Следующий урок

▶



В этом модуле мы поговорим о **Переносе обучения** или **Transfer learning**.

Обучение нейронной сети требует большого объёма данных, например, в ImageNet это миллионы картинок. Что делать, если в текущей задаче не так много данных? Оказывается, в решении новой задачи можно переиспользовать часть нейросети, которая училась на ImageNet.

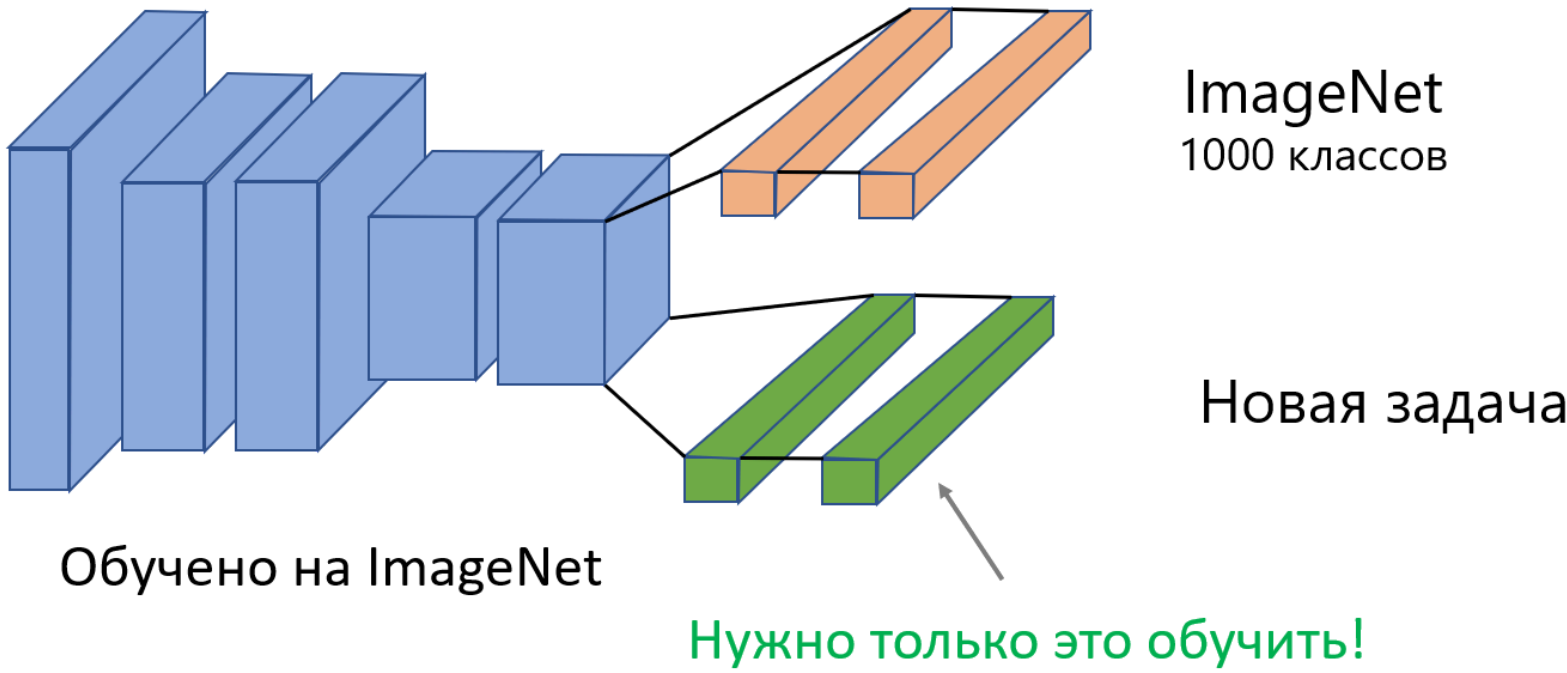
ImageNet — база данных аннотированных изображений, предназначенная для отработки и тестирования методов распознавания образов и машинного зрения. ImageNet использует краудсорсинг для аннотирования изображений.

ImageNet — это dataset, организованный в соответствии с иерархией WordNet.

Сеть, которая обучилась на ImageNet, рассматривают, как состоящую из:

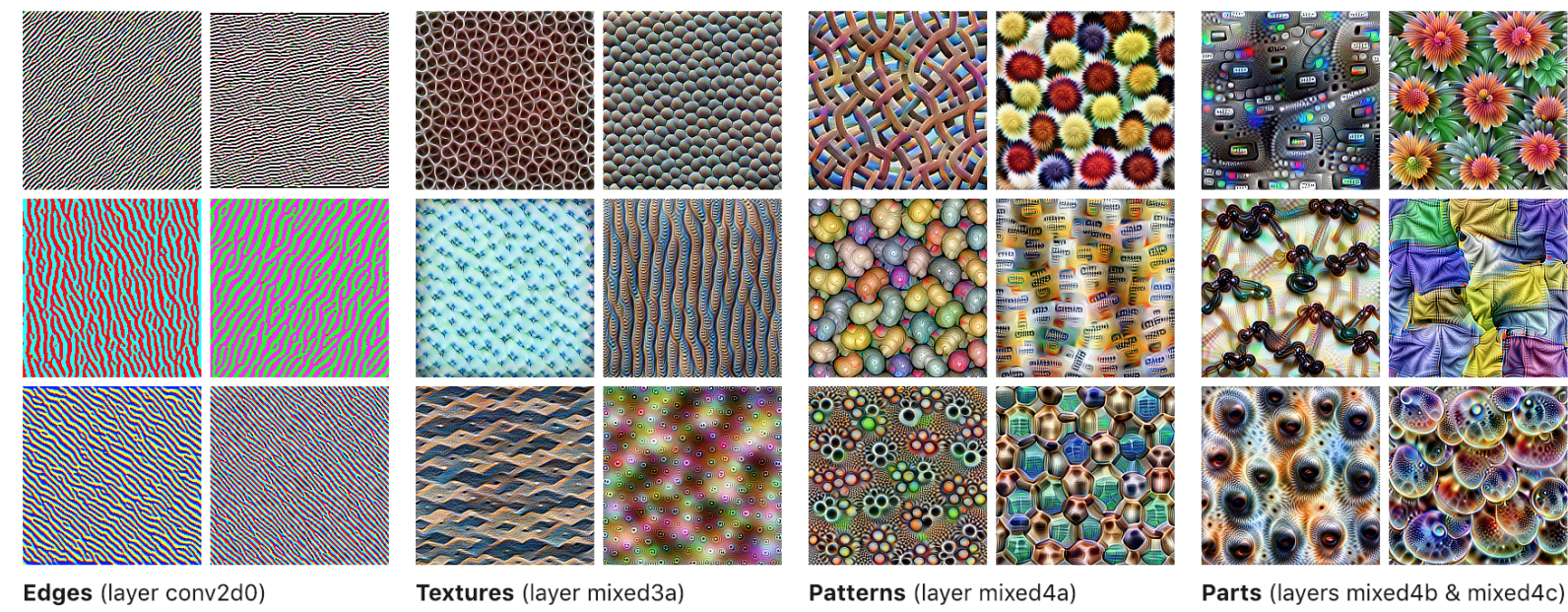
- ▶ первая часть (на рисунке синяя) состоит из сверточных слоёв и извлекает из картинки признаки;
- ▶ вторая часть (на рисунке оранжевая) с помощью многослойного персептрона (MLP) по признакам определяет, к какому из тысячи классов принадлежит картинка.

Экстрактор признаков



Первую (синюю) часть такой сети можно переиспользовать в новой задаче. Такой подход называется Переносом обучения или **Transfer learning**. Таким образом, чтобы обучить сеть на новой задаче, необходимо доучить только крайний полносвязный слой.

Нейроны на разных слоях сети, которую учили на ImageNet, реагируют по-разному:



ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ:

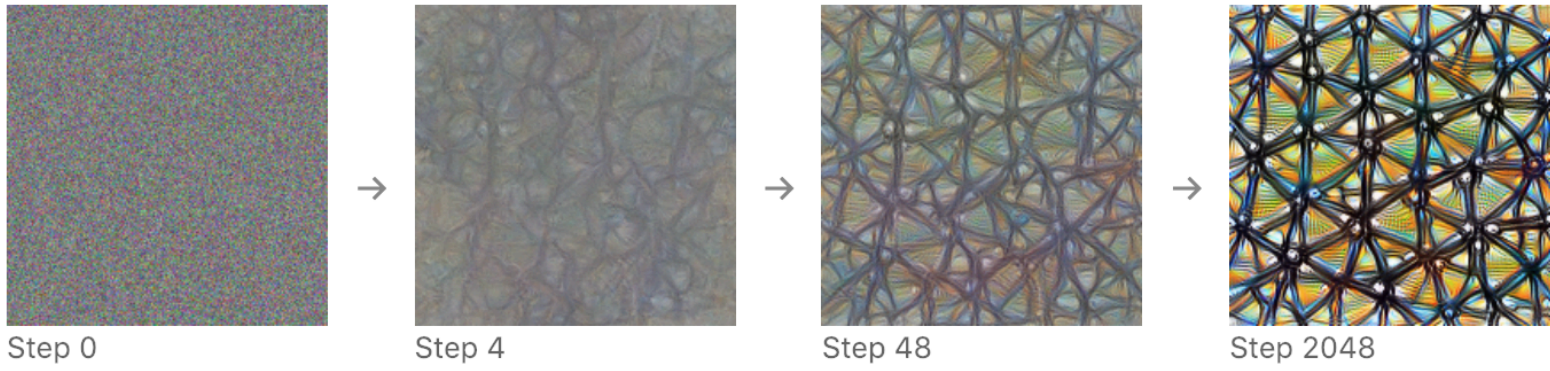
- ▶ Статья «[Feature Visualization. How neural networks build up their understanding of images](#)»;



КАК ПОЛУЧИТЬ КАРТИНКИ, КОТОРЫЕ РИСУЮТ, НА ЧТО РЕАГИРУЮТ НЕЙРОНЫ?

Если вы когда-либо обращались за какими-нибудь услугами, скорее всего вы слышали выражение «Индивидуальный подход к каждому клиенту». Так вот, в работе с нейронами такая же история. Нейронная сеть дифференцируема, каждый нейрон является сложной функцией. Достаточно подобрать вход, который даст максимальную активацию на отдельно взятый нейрон.

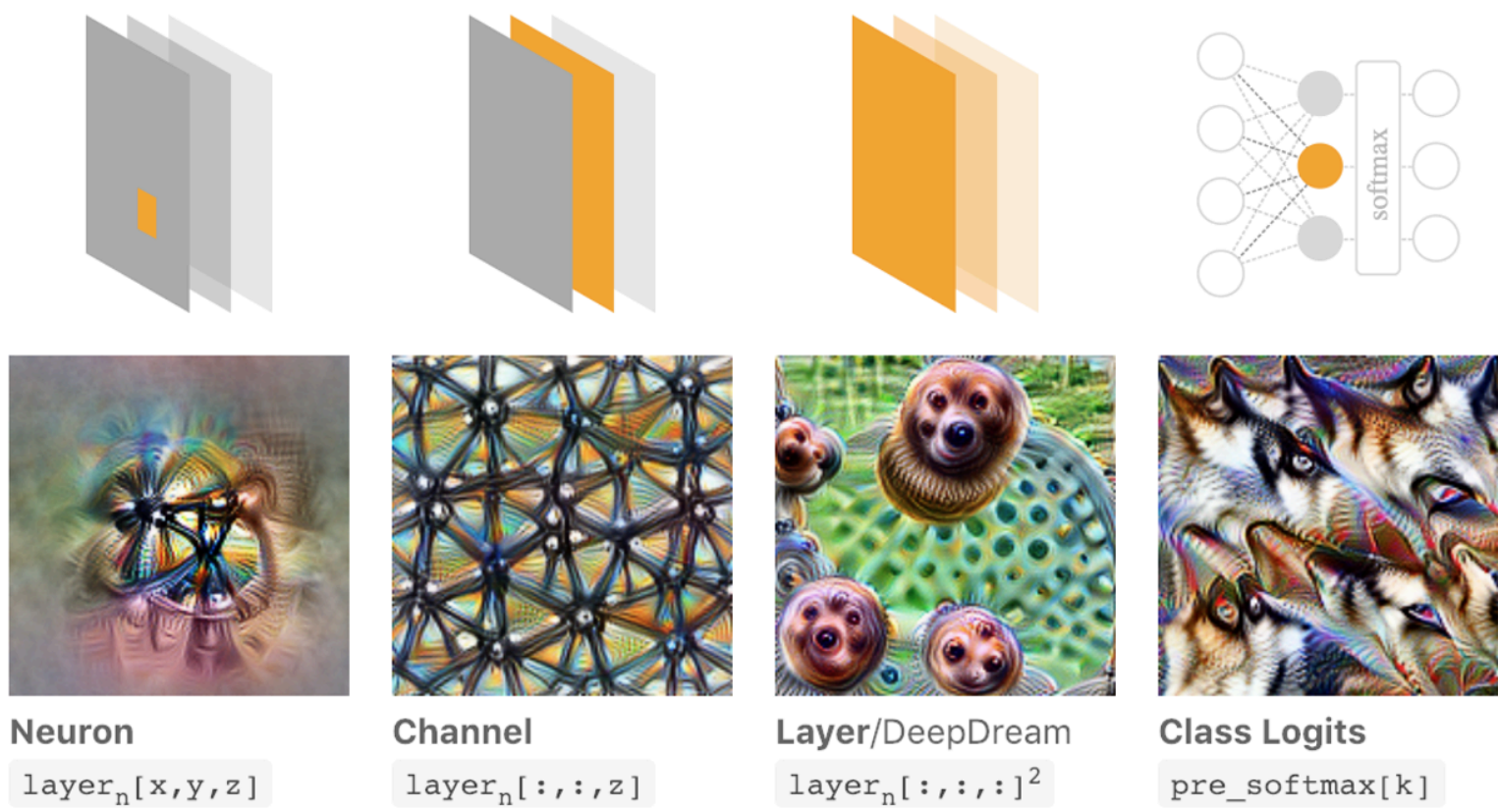
Аргументами функции, которые подбираются с помощью обратного распространения ошибки, является входное изображение. Фактически организуется стохастический градиентный спуск, в ходе которого изображение приобретает осмысленный вид.



Строить такие картинки можно, используя целый слой или канал в слое. Задача: найти, на что отреагирует нейрон, при помощи **метода градиентного спуска (SGD)**.

Если взять на картинке один нейрон, то картинка, которая будет подобрана при помощи SGD, будет содержать кусочек текстуры только в той части изображения, на которую смотрит этот нейрон. Каждый нейрон имеет ограниченное поле обзора. «Слепых пятен» можно избежать, взяв все нейроны слоя.

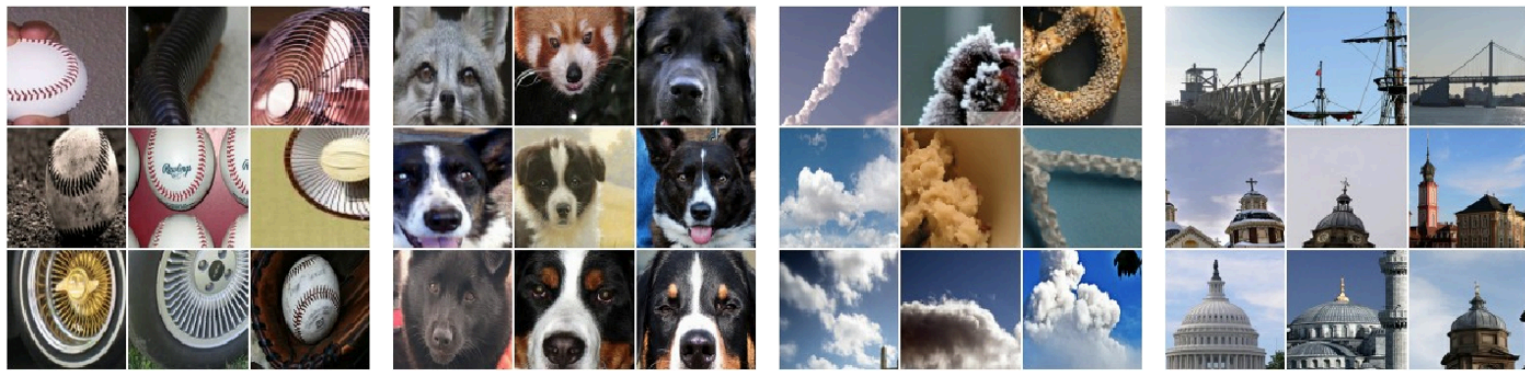
- Можно максимизировать выход одного нейрона, канала, слоя или целого выхода сети для конкретного класса.



- Также можно взять выход нейронной сети, который соответствует конкретному классу, и под него подбирать изображение при помощи SGD.



- Другой способ: в dataset, используемом при обучении, взять картинки, на которых конкретный нейрон даёт максимальную активацию.



В этом случае нужно намного меньше данных для обучения последних слоёв.

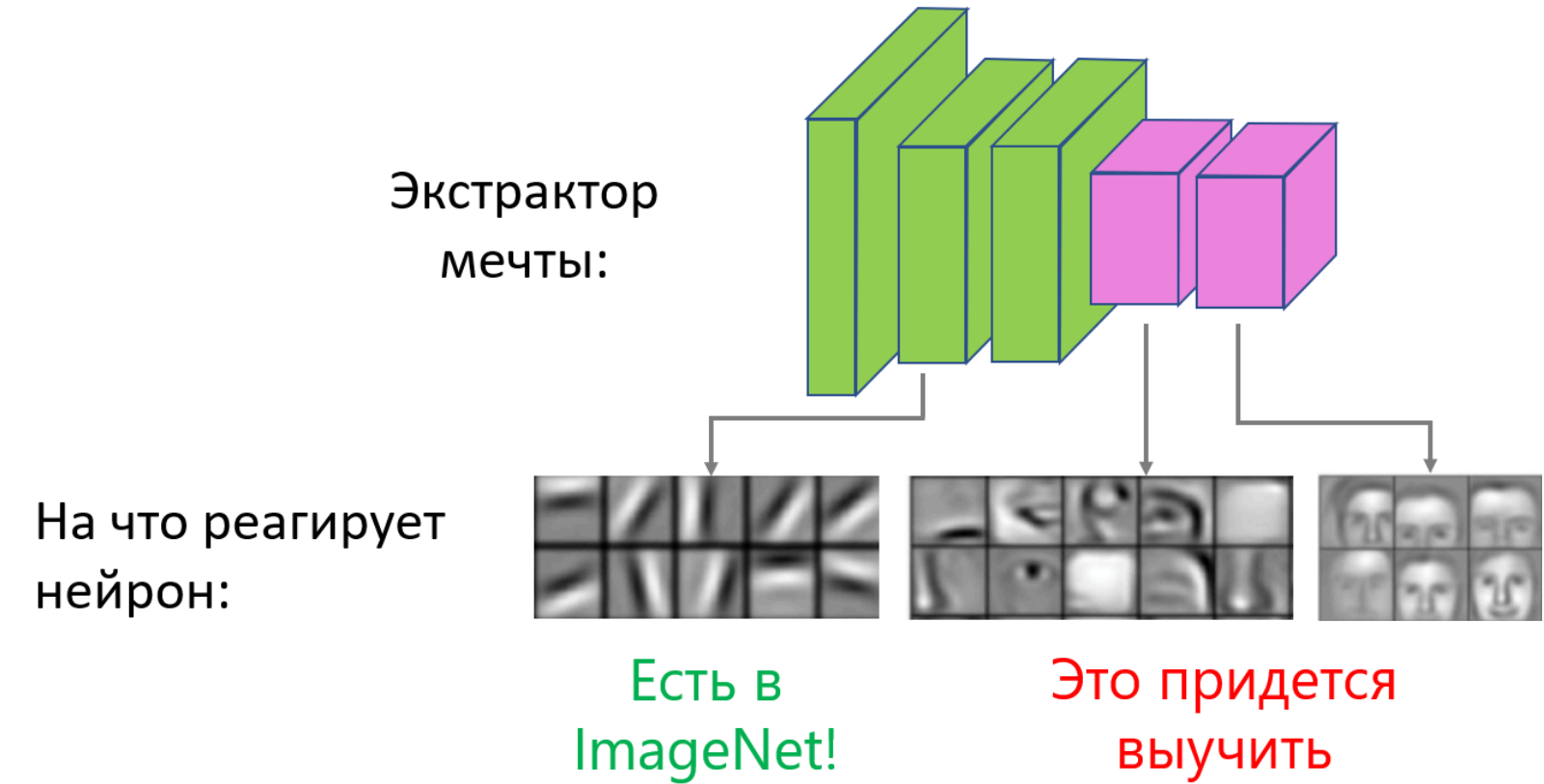
Перенос обучения работает, если новая задача похожа на ImageNet (содержит те же объекты или похожие). Стоит также учесть, что в ImageNet нет лиц.



Разберём задачу распознавания эмоций по фотографии. Если бы нейронная сеть обучалась нами с нуля, то экстрактор выглядел бы следующим образом



В ImageNet сети, наверняка, уже есть детекторы краёв и простые текстуры, которые можно использовать в нашей задаче. Остальные текстуры необходимо будет выучить, используя дополнительные данные для дообучения сети.



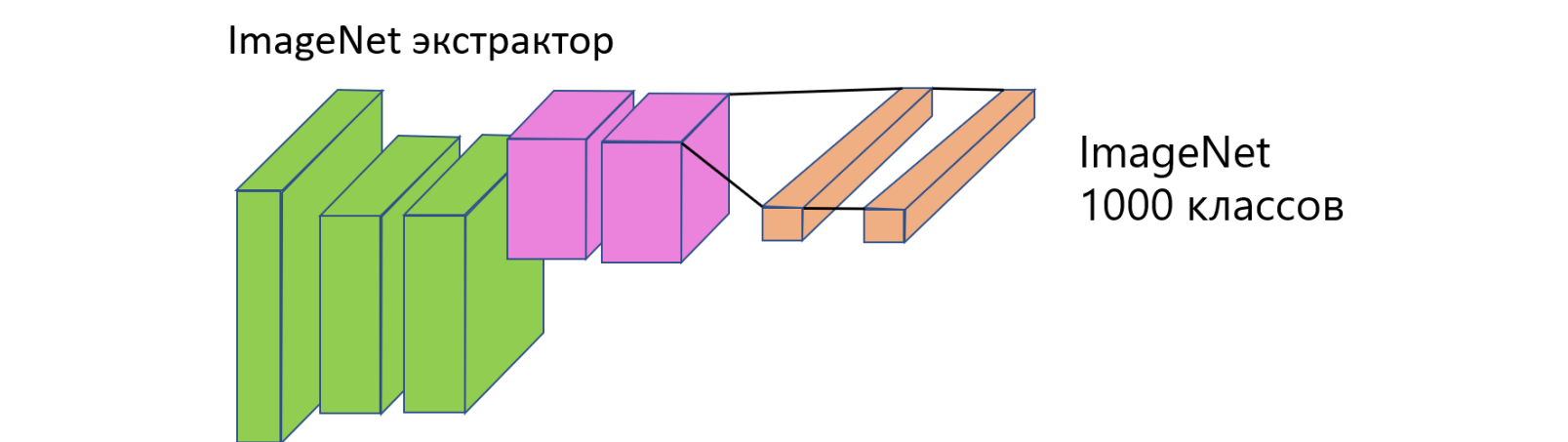
Взяв первые несколько слоёв из фича-экстрактора, который обучен по ImageNet, останется дообучить только последние слои нейронной сети.

< Предыдущий урок

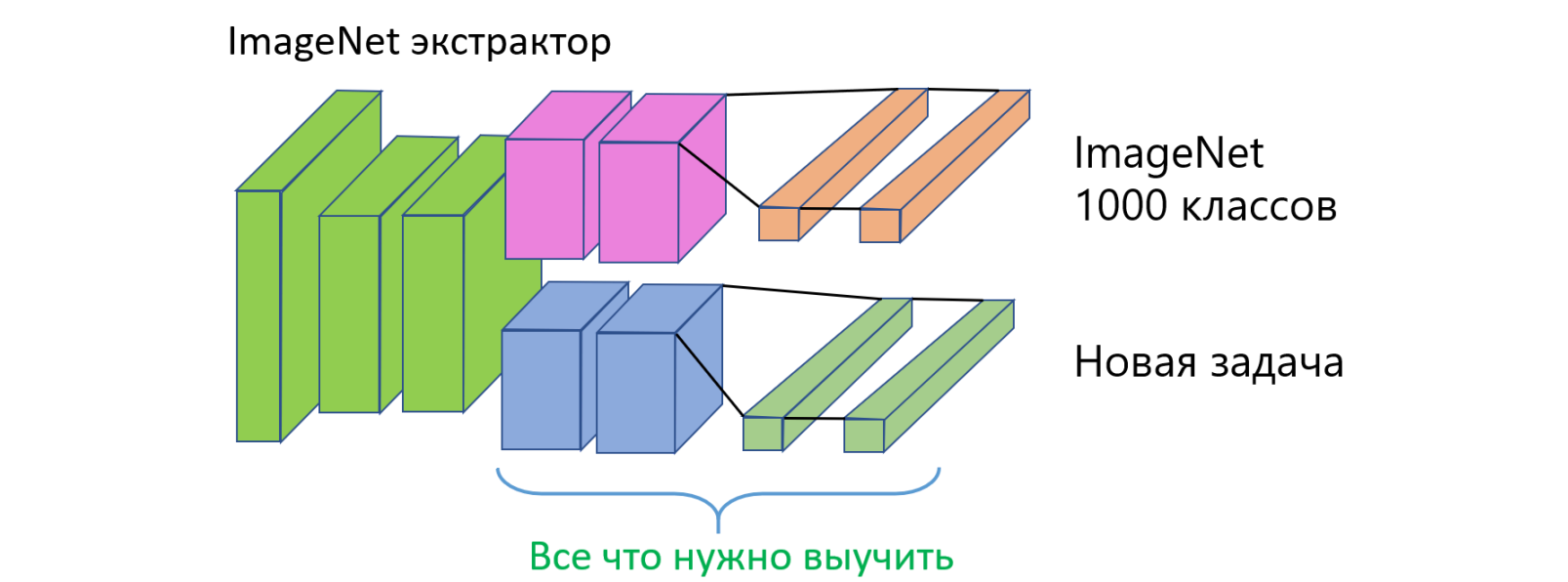
Следующий урок >



Рассмотрим сеть, которая обучена на ImageNet. Фиксируем в ней часть фича-экстрактора (зелёная на рисунке).

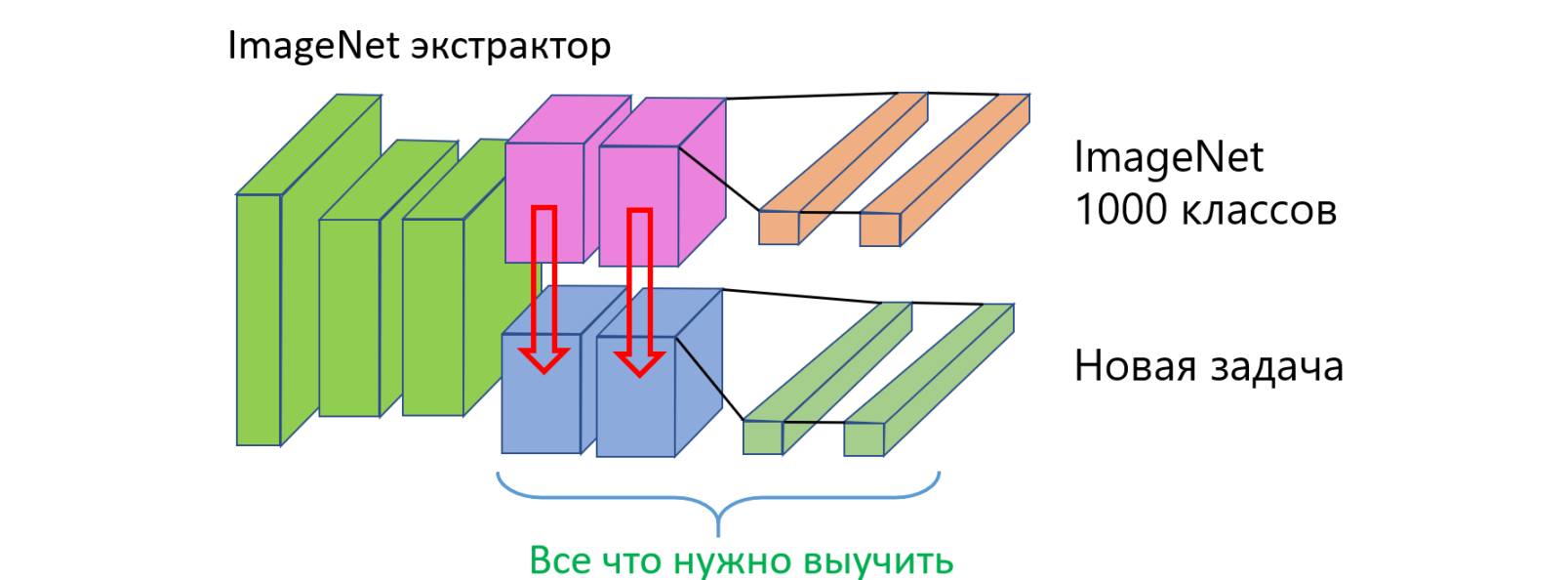


Тогда в новой задаче дообучить предстоит только синий и тёмно-зелёный слои.



Здесь используется **Fine-tuning** — связный приём, очень похожий на Transfer learning. Заключается он в следующем:

Синие свёрточные слои будем инициализировать не случайными значениями, а теми же, что использовались в ImageNet сети. Тёмно-зелёный слой зафиксируем, а синий — инициализируем теми же весами, которые получили от ImageNet сети. И пустим градиенты с маленьким шагом, чтобы подстроить слои под нашу задачу.



ПРЕИМУЩЕСТВА FINE-TUNING

- FINE-TUNING** — Это очень популярный подход, так как:
- ImageNet покрывает огромное количество объектов, которые есть в реальном мире.
 - В Keras есть веса для обученных на ImageNet VGG, Inception, ResNet и т.д.
 - Можно легко сделать ансамбль из этих сетей.

Резюмируя, обратимся к следующей таблице:

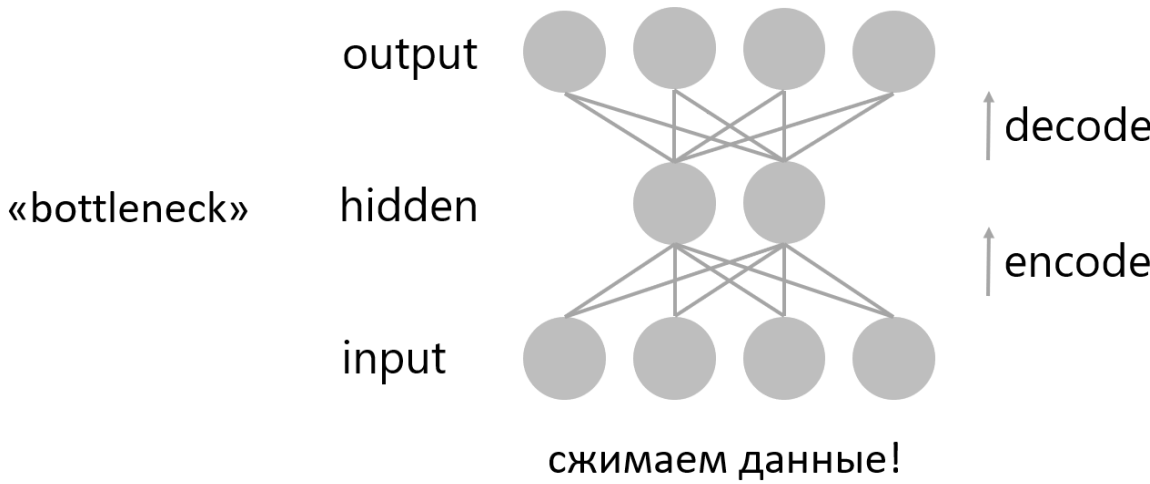
	Похоже на ImageNet	Не похоже
Мало данных	Учим последние слои	Собираем больше данных
Много данных	Fine-tuning большего числа слоев	Учим с нуля

SKILLFACTORY

Автокодировщики (autoencoders) — это специальная архитектура нейронной сети, позволяющая сжимать входные данные. Автокодировщик сжимает входные признаки в более компактное представление, которое не теряет информации об исходных данных.

Encoder сжимает признаковое описание на входе, decoder расшифровывает эту информацию обратно, не изменяя входные данные:

- ▶ Encoder = data to hidden
- ▶ Decoder = hidden to data
- ▶ $\text{Decoder}(\text{Encoder}(x)) \sim x$ (близость по MSE)



Такое сжатое представление или **bottleneck** (горлышко бутылки) будет полезным для решения задач.

ВСПОМНИМ ПРО РСА (МЕТОД ГЛАВНЫХ КОМПОНЕНТ)

Этот метод основан на представлении исходной матрицы признаков в виде произведения других матриц и позволяет найти пространство меньшей размерности, хорошо описывающее входные данные. Этот метод в следующем модуле будет разобран подробно.

X

=

U

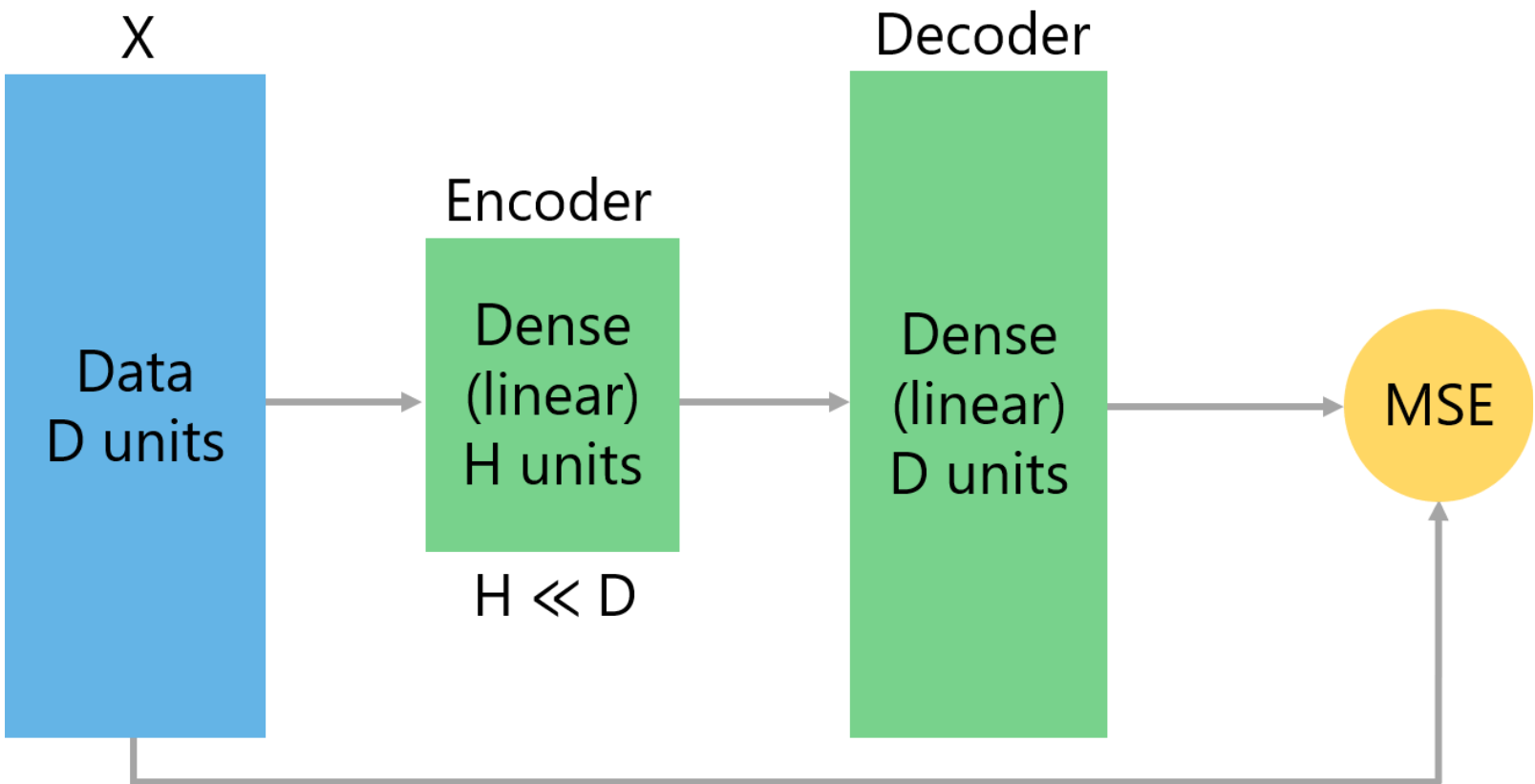
×

V^T

$\|X - U \cdot V^T\| \rightarrow \min_{U, V}$

Такую задачу можно переписать в виде нейронной сети:

- ▶ **D признаков** на входе при помощи полносвязного слоя **Encoder** сжимаем до представления в **H признаков**.
- ▶ **H** много меньше **D**.
- ▶ Полносвязный **слой Decoder** с помощью маленького представления **H** пытается восстановить исходные **D** признаков.
- ▶ **Цель** обучения такой нейросети в том, чтобы представление **H**, пропущенное через сеть, не изменяло входные данные.

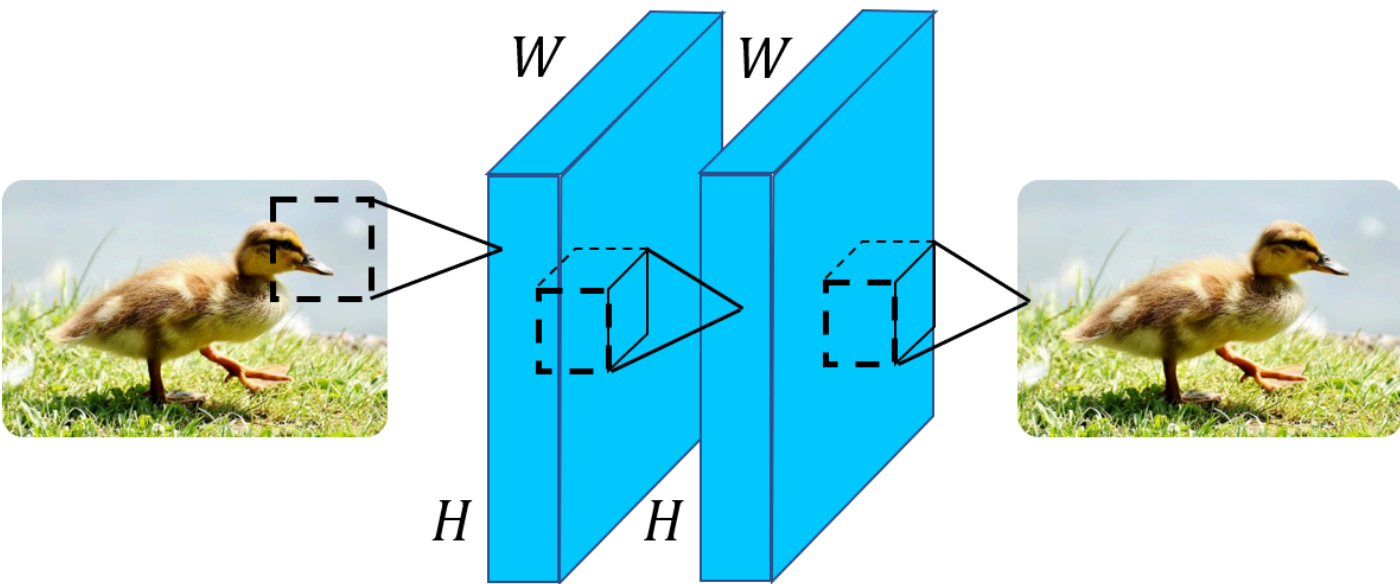


Метод главных компонент позволяет сжимать картинку в 10 раз.



Результат предыдущего метода можно улучшить. Ведь для изображений применяются свёрточные нейронные сети.

Добавим свёрточные слои. К исходной картинке применим свёрточный слой, описывая каждый пиксель изображения по пикселям, которые были в окрестности исходного изображения. Добавим ещё несколько слоёв.

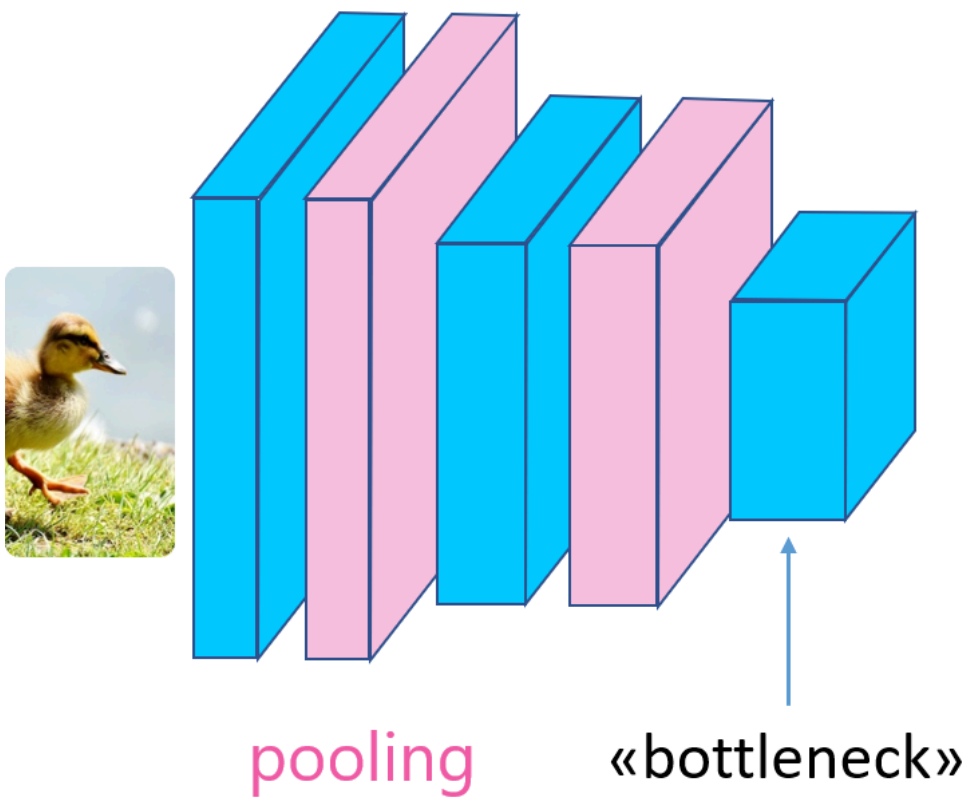


Попиксельный MSE

На выходе предстоит «схлопнуть» представление в картинку исходного расширения с одним или тремя (в случае цветного изображения) выходными каналами.

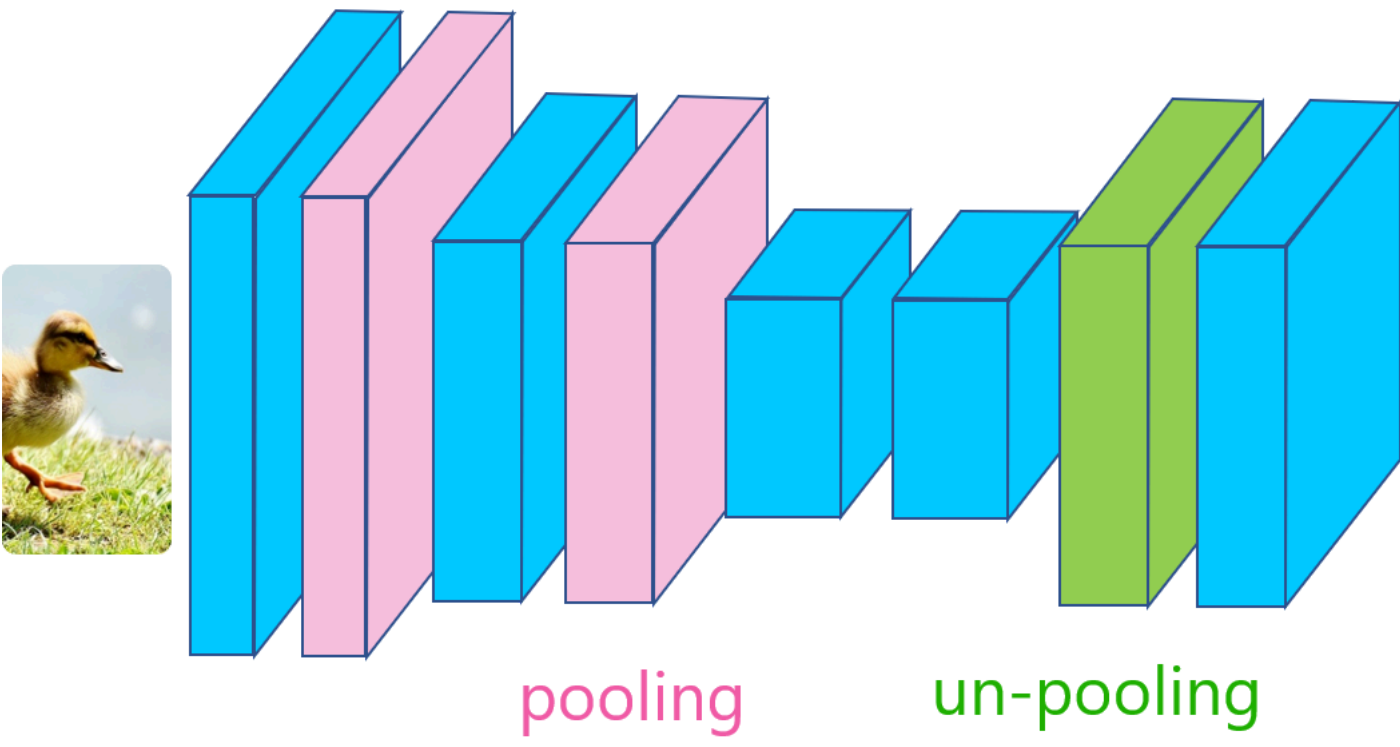
При помощи среднеквадратичных потерь можно обучить это преобразование. Но это достаточно наивный подход, так как требует слишком много вычислений и не содержит bottleneck.

Чтобы получить компактное представление в таких сетях, добавим pooling слой.



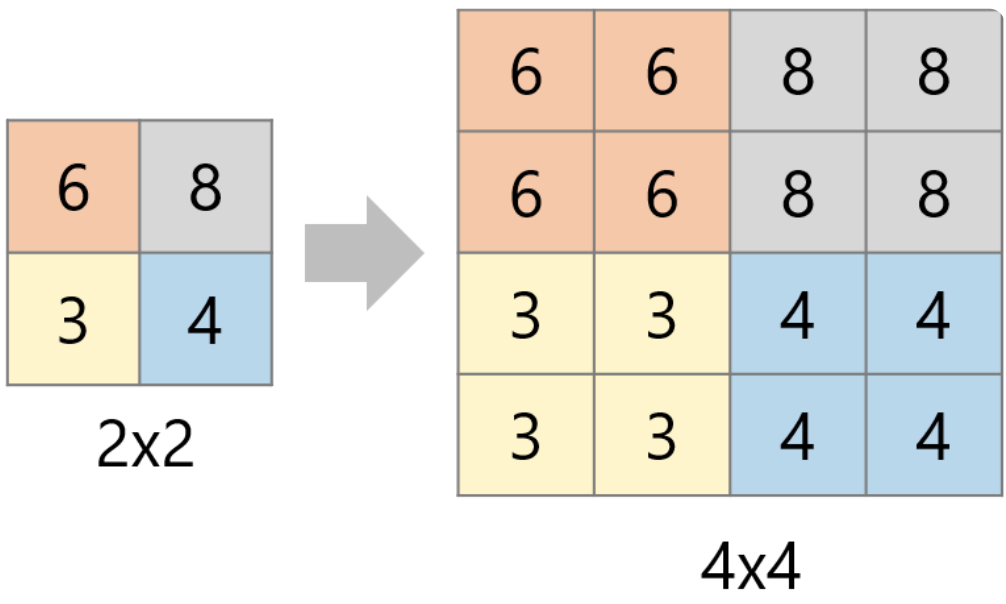
Pooling слой будет сжимать изображение в два раза. На выходе получим небольшой тензор, который можно вытянуть в вектор, — это и будет представлением для картинки. Это представление будем называть **embedding** или результатом **encoder**’а.

Как развернуть полученный вектор из числа обратно в картинку? Кажется естественным повторить преобразование **encoder**’а в обратную сторону. Как обратить результат pooling слоя? Используем **un-pooling** слой.

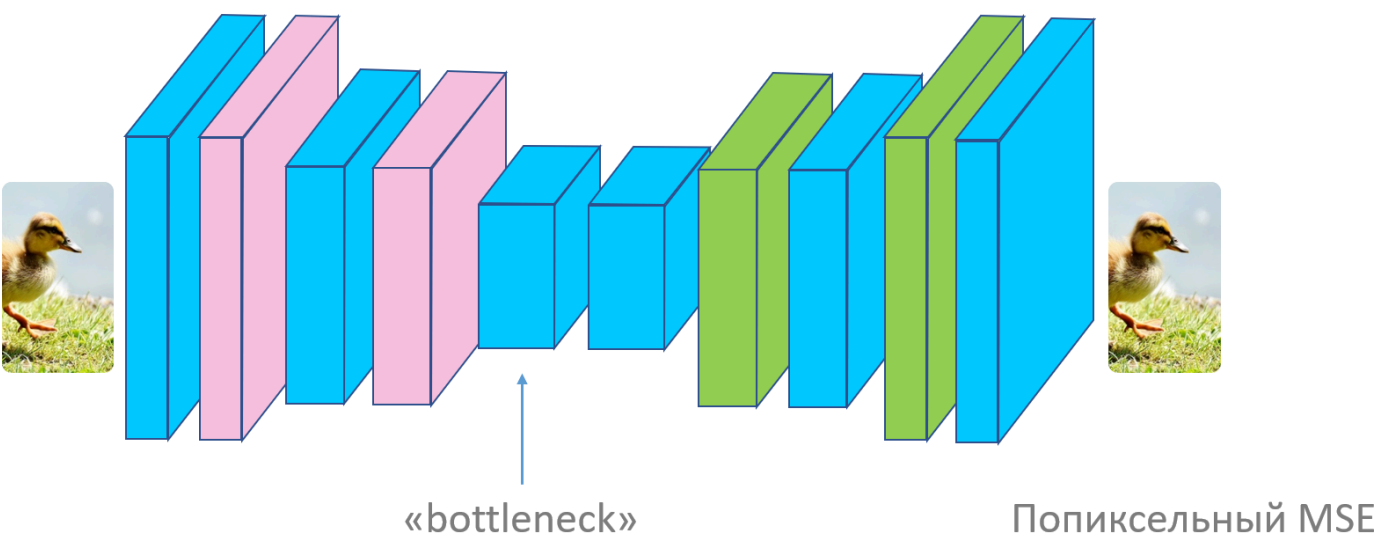


UN-POOLING

Рассмотрим простой вариант. Будем делать **un-pooling** при помощи заполнения ячеек теми же значениями, которые были рядом, интерполируя значения ближайшего соседа.



Такой простой подход работает. Свёрточные слои после **un-pooling** сгладят «плохие» преобразования. Всю эту сеть можно обучить при помощи средней квадратичной ошибки MSE.



«bottleneck»

Попиксельный MSE

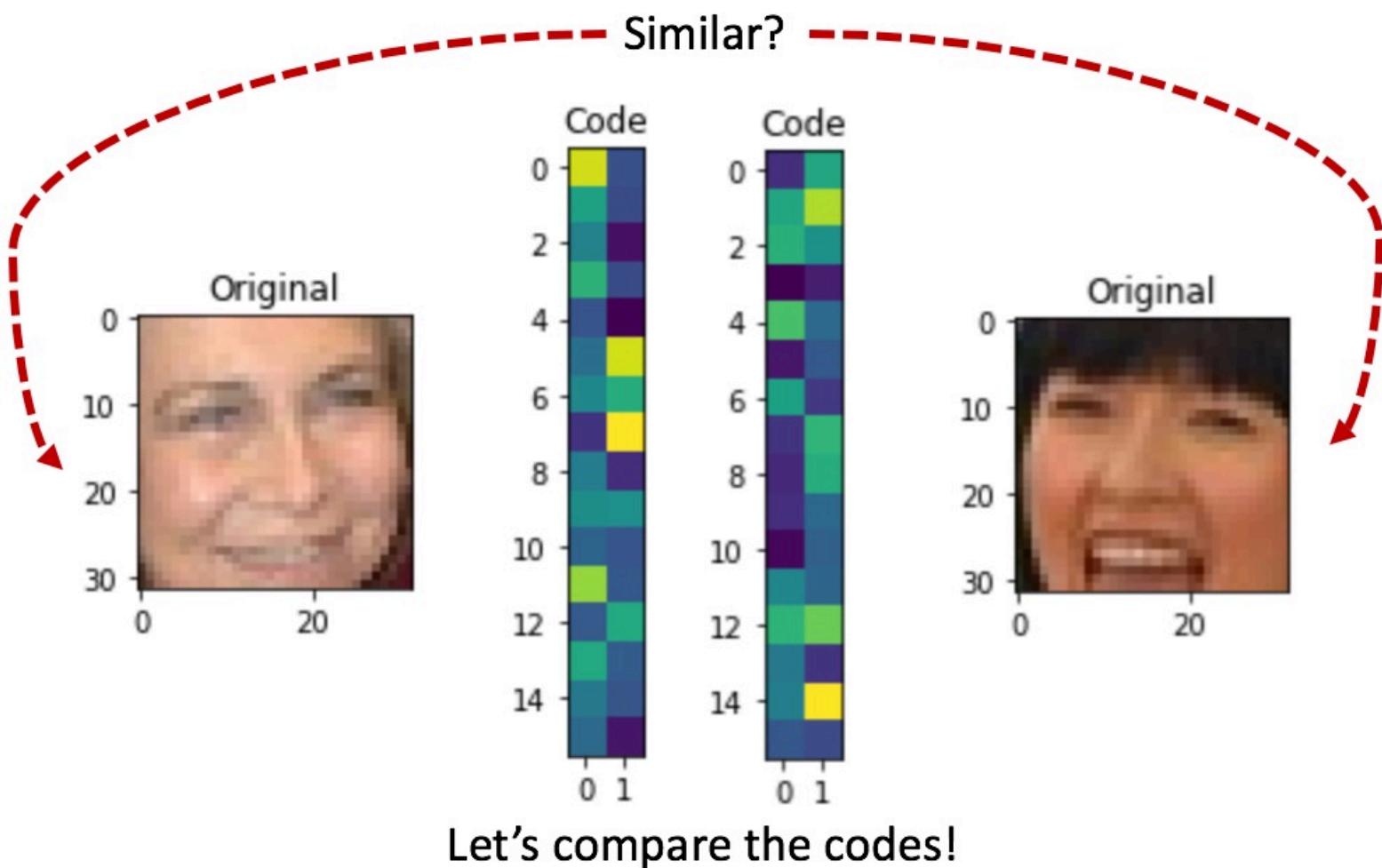
7/17

Поиск похожих изображений

текущий урок

SKILLFACTORY

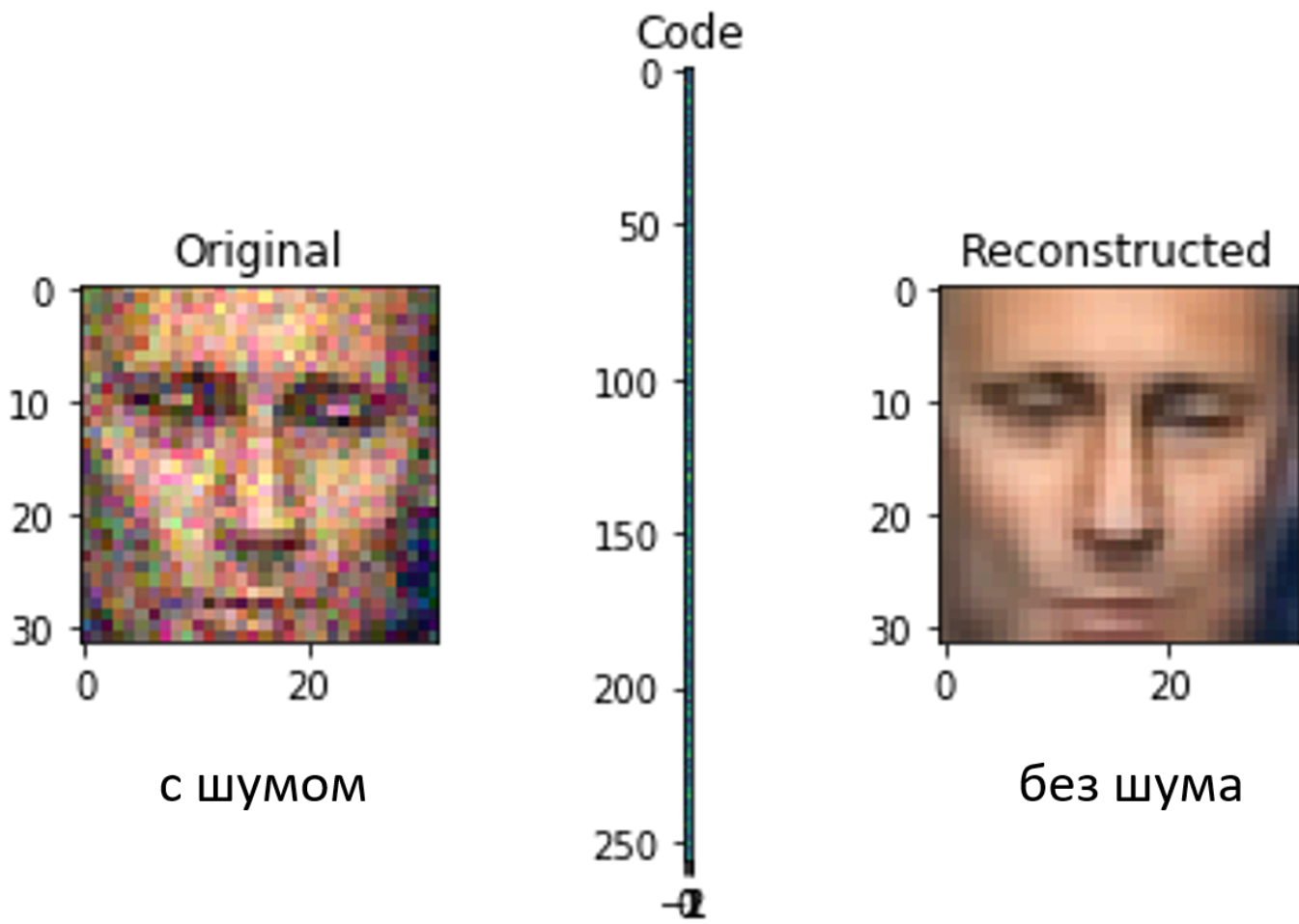
Пусть нам нужно сравнить два изображения.



Понятно, что на картинках может быть один и тот же объект, но с разных ракурсов. Сравнивать попиксельно невозможно, необходимы признаки более высокого уровня, которые может дать конволюционная сеть.

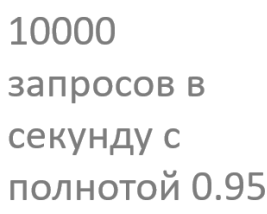
Возьмём большое количество таких лиц и прогоним через autoencoder. Результат работы encoder'a (bottleneck) для каждого изображения запоемнм, как вектор. Поиск похожих изображений будет заключаться в поиске похожих векторов. И это работает!

Denoising autoencoders — ещё одно применение autoencoders. Denoising autoencoders учатся удалять шум с изображения.



← Предыдущий урок

Следующий урок →

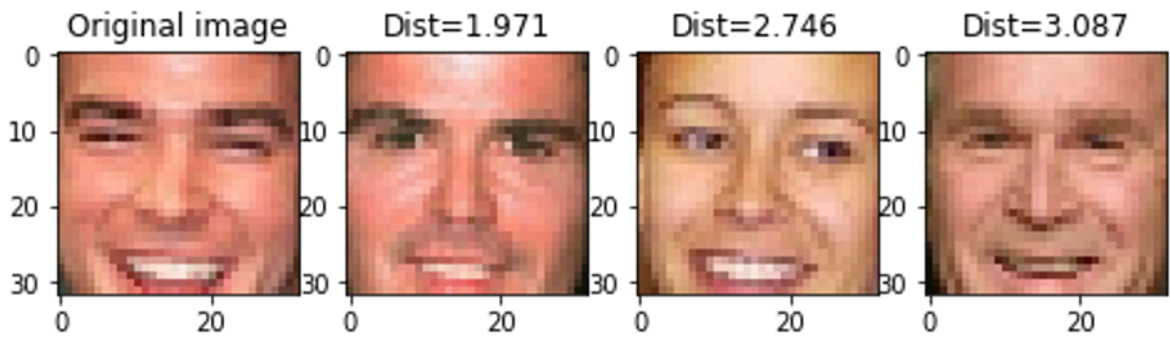


- «Benchmarking nearest neighbors»
- «Code for reproducing 2nd place solution for Kaggle competition IEEE's Signal Processing Society»

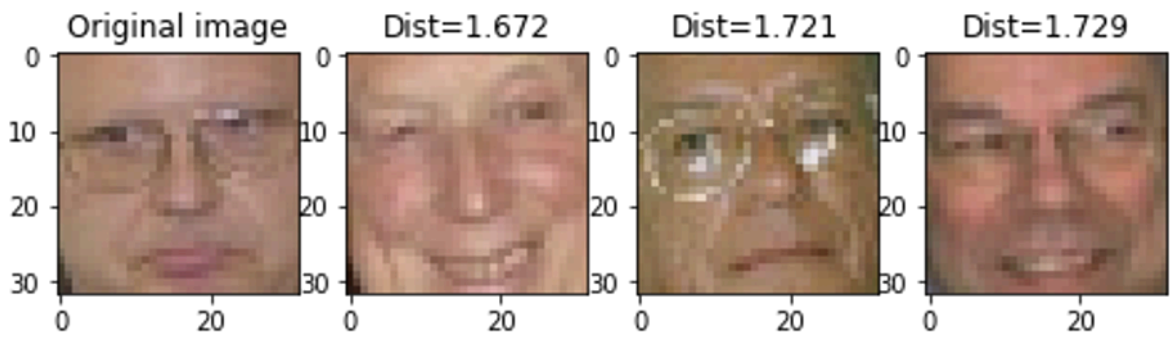


РАССМОТРИМ НА ПРИМЕРЕ, КАК РАБОТАЕТ ПОИСК ПОХОЖИХ ИЗОБРАЖЕНИЙ:

Возьмём картинку, на которой человек улыбается, превратим её в вектор и найдём ближайшие по близости векторов картинки. Окажется, что на ближайших картинках изображены лица людей, которые имеют схожий ракурс и улыбку.



Другой пример: для не улыбающегося лица в очках найдутся следующие похожие картинки:



В пространстве embedding'ов мы можем делать интерполяцию, плавно меняя картинку



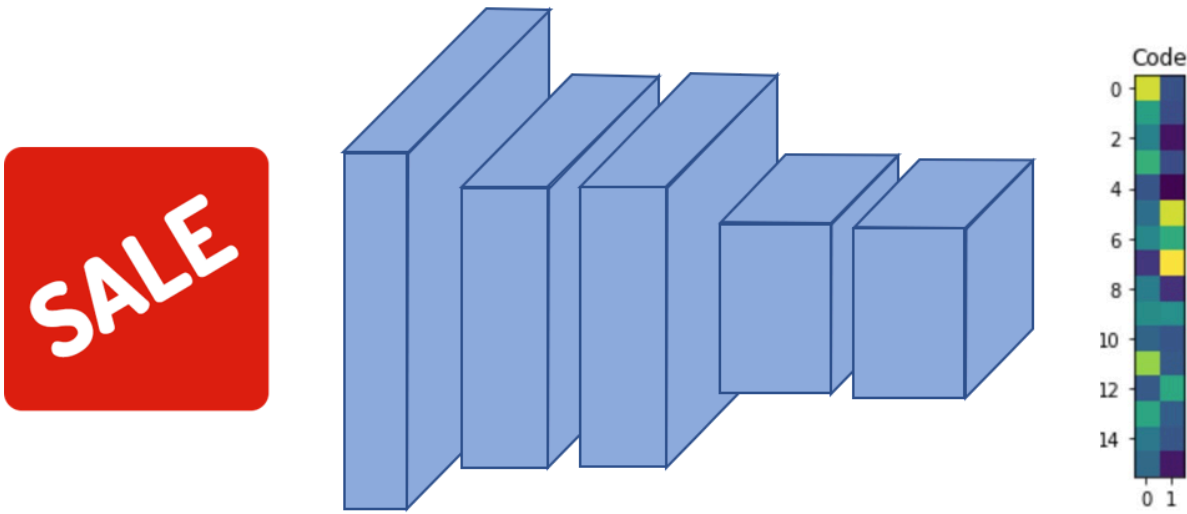


10/17 Вывод



Для чего нужны autoencoder'ы?

- ▶ Нам не нужны размеченные данные! Учим экстрактор признаков за бесплатно!
- ▶ Полученный экстрактор можно файн-тьюнить под нашу задачу!



В этом модуле мы рассмотрели два важных подхода к нейросетевым представлениям:

1. **Fine-tuning / Transfer learning**, в котором смотрим на часть нейросети, как на экстрактор признаков для нашей задачи. Этот экстрактор мы можем полностью или частично переиспользовать.
2. Второй подход заключается в использовании неразмеченных данных, где учим autoencoder и его encoder-часть берём как экстрактор признаков.

[← Предыдущий урок](#)

[Следующий урок >](#)



```
File Edit Selection View Go Run Terminal Help
transfer_learning_keras_flowers_sf_dl_v2.ipynb
transfer_learning_keras_flowers_sf_dl_v2.ipynb > MTTA > test_generator = test_datagen.flow_from_dataframe(
+ Code + Markdown | Run All | Restart | Clear All Outputs | Variables | Outline ...
Python 3.11.1 undefined

jpg/image_07420.jpg 93 93
jpg/image_01122.jpg 45 45

print('Accuracy: %.2f%%' % (accuracy_score(tmp_y['Category'], tmp_y['Predict'])*100))
[102] ✓ 0.0s Python
... Accuracy: 94.93%

TTA

model.load_weights('best_model.hdf5')
[103] ✓ 0.0s Python

> test_datagen = ImageDataGenerator(rescale=1. / 255,
rotation_range = 90,
shear_range=0.2,
zoom_range=[0.75,1.25],
brightness_range=[0.5, 1.5],
width_shift_range=0.1,
height_shift_range=0.1,
[111] ✓ 0.0s Python
```

1x 1:20 / 3:36

↓ Ноутбук к скринкасту



Полезные ссылки:

- ▶ [Paperswithcode](#)
- ▶ [efficientnet на GitHub](#)

← Предыдущий урок

Следующий урок →

текущий урок



- ▶ A Comprehensive guide to Fine-tuning Deep Learning Models in Keras (Part I) // flyyufelix.github.io [Электронный ресурс]. URL: <https://flyyufelix.github.io/2016/10/03/fine-tuning-in-keras-part1.html> (дата обращения: 10.03.2024).
- ▶ Benchmarks of approximate nearest neighbor libraries in Python // github.com [Электронный ресурс]. URL: <https://github.com/erikbern/ann-benchmarks> (дата обращения: 10.03.2024).
- ▶ Code for reproducing 2nd place solution for Kaggle competition IEEE's Signal Processing Society - Camera Model Identification // github.com [Электронный ресурс]. URL: <https://github.com/ikibardin/kaggle-camera-model-identification> (дата обращения: 10.03.2024).
- ▶ Implementation of EfficientNet model. Keras and TensorFlow Keras // github.com [Электронный ресурс]. URL: <https://github.com/qubvel/efficientnet> (дата обращения: 10.03.2024).
- ▶ Browse State-of-the-Art// paperswithcode.com [Электронный ресурс]. URL: <https://paperswithcode.com/sota> (дата обращения: 10.03.2024).
- ▶ Feature Visualization // distill.pub [Электронный ресурс]. URL: <https://distill.pub/2017/momentum/> (дата обращения: 10.03.2024).

Следующий урок >